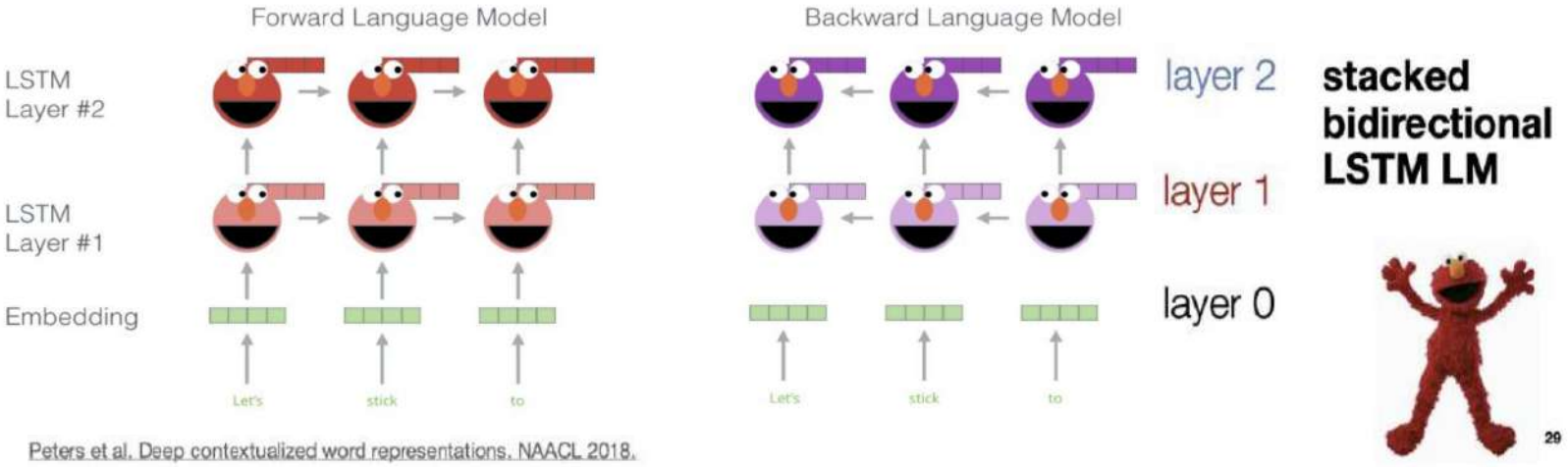
Natural Language Processing

UNIVERSITÄT BONN

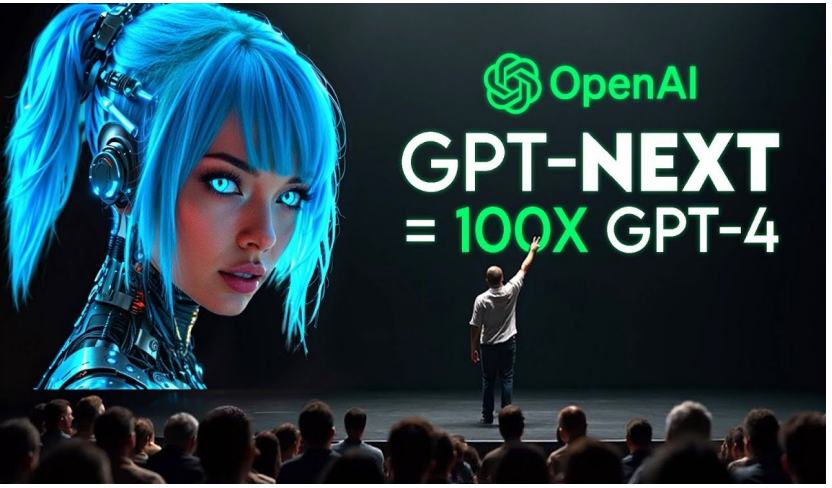


Peters et al. Deep contextualized word representations. NAACL 2018.

Lecture 3: Recurrent NNs, ELMo

Content Overview





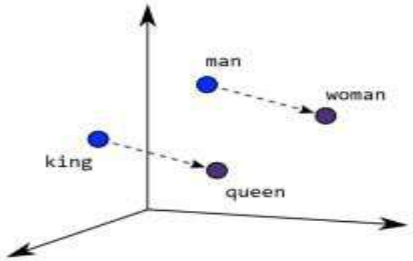
- 30.10. Intro, types of NLP tasks, preprocessing
- 6.11. Word vectors, embeddings
- 13.11. Neural nets - RNNs, attention, ELMo
- 20.11. Transformer overall, BERT
- **27.11.** BERT continued (specifics), BPE
- 4.12. Bertology, GPT2, T5
- **11.12.** LLM Evaluation
- 18.12. Decoding strategies
- 8.1. from "Transformers" to "LLMs", instruction tuning, reasoning, alignment
- 15.1. Multilingual/-modal LLMs, agents
- 22.1. Ongoing research topics
- 29.1. Last lecture - repetition for the exam

Recap from last time

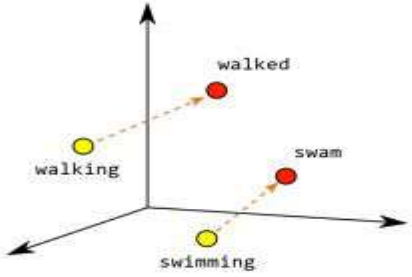
- Word vectors, word embeddings
- Word2vec and its variants
- Dealing with some limitations - FastText
  - Representing unknown words

Some limitations still remain

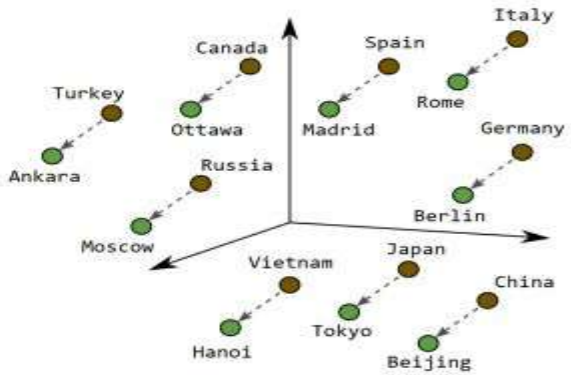
  - multiple senses, context-dependent meaning…



Male-Female

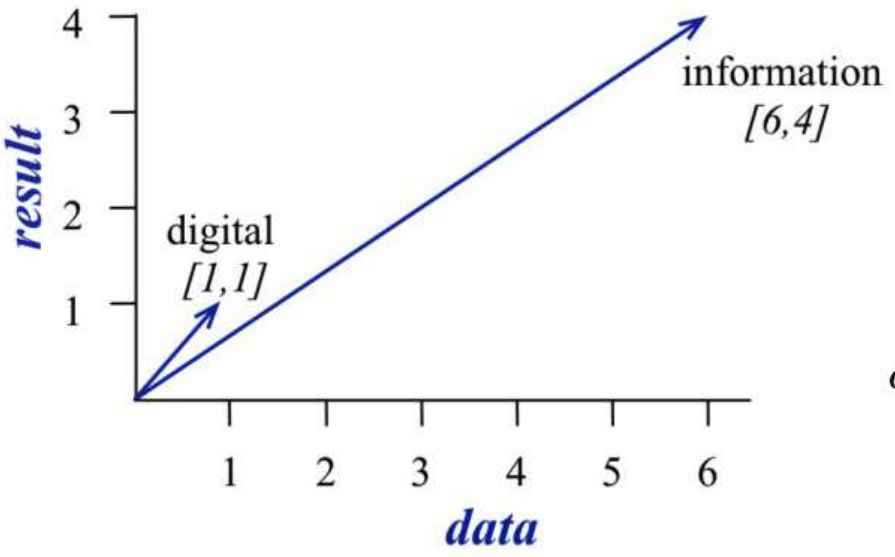


Verb Tense



Country-Capital

Words as vectors: sparse to dense, cosine simil.



$$cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|}$$

$$cos(\mathbf{u}, \mathbf{v}) = \frac{\sum_{i=1}^{V} u_i v_i}{\sqrt{\sum_{i=1}^{V} u_i^2}\sqrt{\sum_{i=1}^{V} v_i^2}}$$

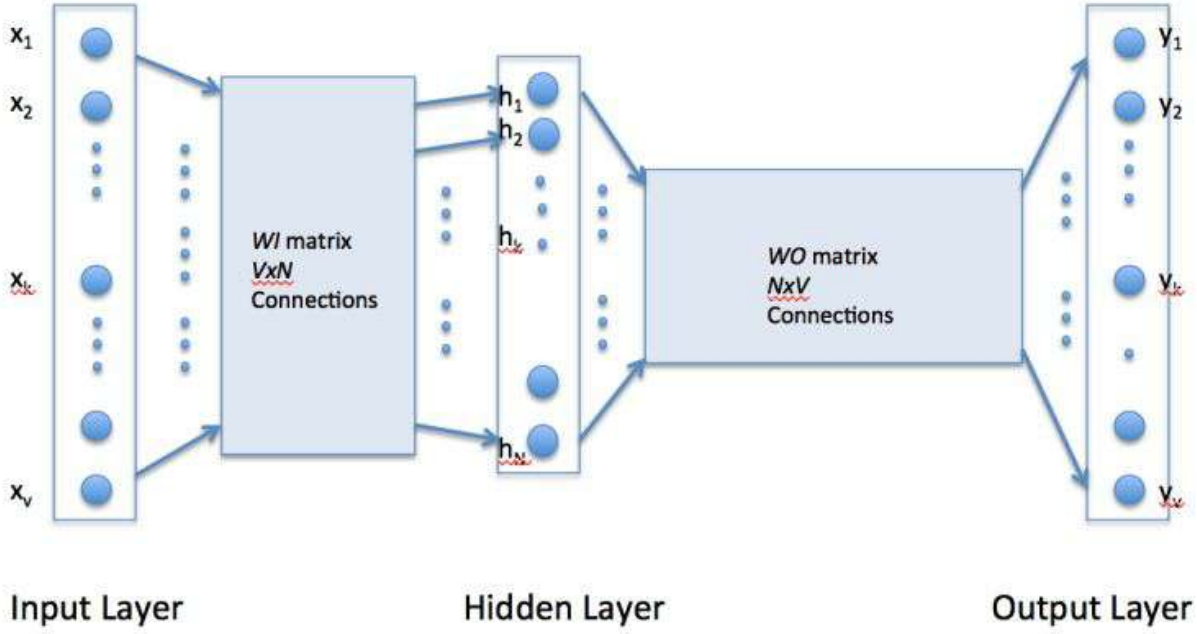
$$\text{employees} = \begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 10.109 \\ -0.542 \\ 0.349 \\ 0.271 \\ 0.487 \end{pmatrix}$$



Word2vec training

- turning the internet to a supervised classification task with a DNN
- secret key: dot product

positive examples +		negative examples -			
t	c	t	c	t	c
apricot	tablespoon	apricot	aardvark	apricot	seven
apricot	of	apricot	my	apricot	forever
apricot	jam	apricot	where	apricot	dear
apricot	a	apricot	coaxial	apricot	if



"softmax"

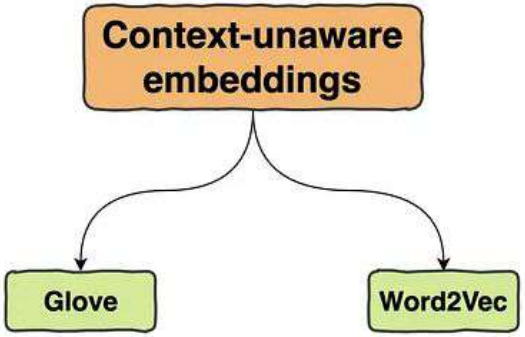
$$P(w_{t+j} \mid w_t) = \frac{\exp(\mathbf{u}_{w_t} \cdot \mathbf{v}_{w_{t+j}})}{\sum_{k \in V} \exp(\mathbf{u}_{w_t} \cdot \mathbf{v}_k)}$$

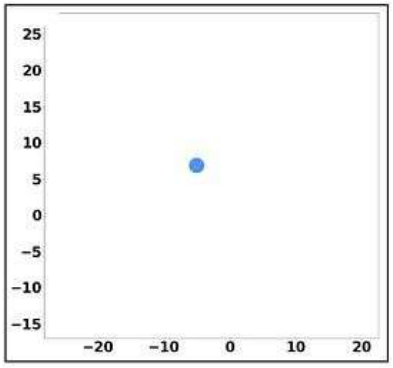
Today's goals



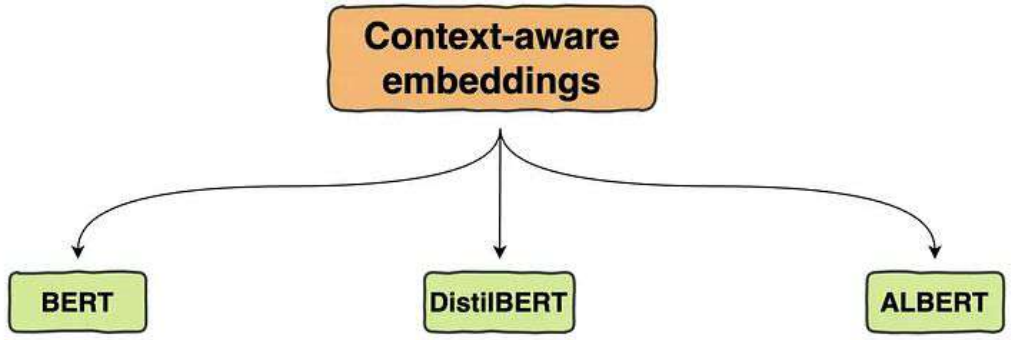
- Leftover from Lecture 2 - intro to BPE
- Language Modeling as a task
- Neural LMs
- Why RNNs are great as neural LMs
- ELMo as a groundbreaking neural LM in 2017
- Limitations of RNNs, even LSTMs
- Introduction to attention (to be continued)

Context-aware Embeddings

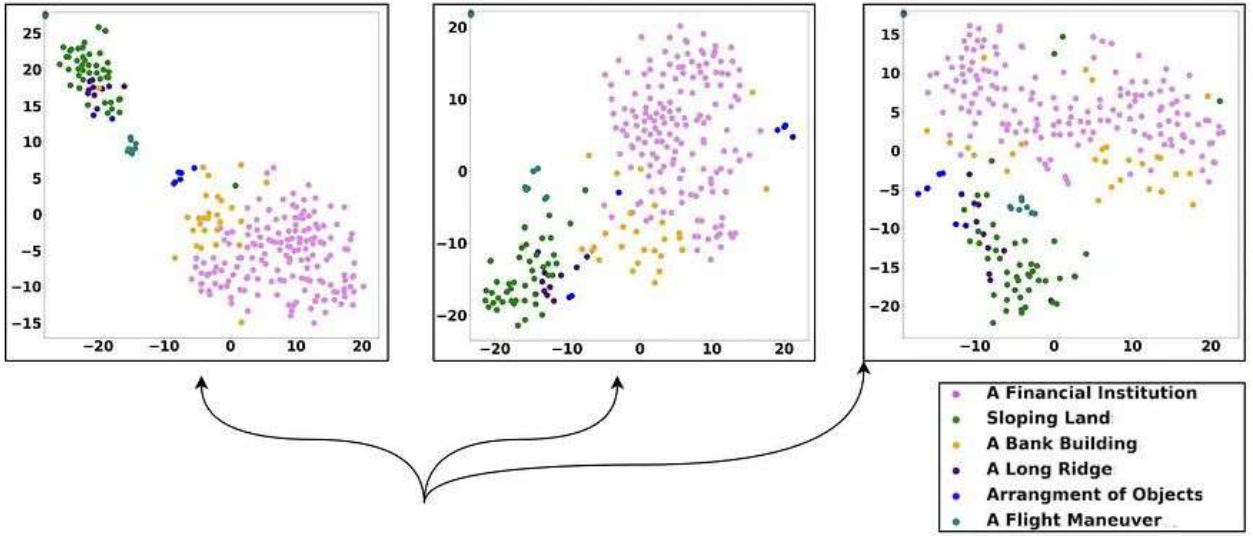




All different meanings of a word get the same representation



Visualization of the embeddings obtained for the word "Bank" in different contexts



Model distinguishes between different meanings of a word

Data-Driven Segmentation

- Most popular methods:
  - Byte pair encoding (BPE)
  - SentencePiece's unigram LM

SentencePiece



SentencePiece is an unsupervised text tokenizer and detokenizer mainly for Neural Network-based text generation systems where the vocabulary size is predetermined prior to the neural model training. SentencePiece implements **subword units** (e.g., **byte-pair-encoding (BPE)** [Sennrich et al.]) and **unigram language model** [Kudo.]) with the extension of direct training from raw sentences. SentencePiece allows us to make a purely end-to-end system that does not depend on language-specific pre/postprocessing.

This is not an official Google product.

Technical highlights

Data-Driven Segmentation

- Most popular methods:
  - Byte pair encoding (BPE)
  - SentencePiece's unigram LM

- these are easy and fast to use & work well

- they permit unbounded vocabularies with a relatively small number of parameters

Byte Pair Encoding (BPE)

(Gage, 1994)

- simple data compression technique
- iteratively replaces most frequent pair of bytes in a sequence with a single, unused byte
- Sennrich et al. (2016) adapted BPE for characters and character sequences

Byte Pair Encoding (BPE)

- **merge**: a rule that combines two consecutive units into a single unit
- initially, units are characters
- after merges, units become character sequences
- greedy algorithm:
  - merge two units with the largest unit bigram count, produce merged unit
  - replace all instances of that 2-unit sequence with the merged unit, recompute counts

Byte Pair Encoding (BPE)

- Sennrich et al. use BPE based on word counts from a corpus
  - sentences are not used; all that's needed are word types and their counts
  - special treatment for end-of-word symbol </w> (an unseen initial step merges final character in each word with </w>)
  - when segmenting new data, segments words individually (does not use context)

Corpus:

```
low
low
lower
lowest
high
high
higher
```

Merges:

Sennrich et al. (2016): *Neural Machine Translation of Rare Words with Subword Units*

Corpus:

```
low</w>
low</w>
lower</w>
lowest</w>
high</w>
high</w>
higher</w>
```

Merges:

actually the corpus looks like this

Sennrich et al. (2016): *Neural Machine Translation of Rare Words with Subword Units*

Corpus:

```
low</w>
low</w>
lower</w>
lowest</w>
high</w>
high</w>
higher</w>
```

Merges:

```
w </w>   (2)
r </w>   (2)
h </w>   (2)
t </w>   (1)
```

the first thing we do is merge word-ending characters with </w>

Sennrich et al. (2016): *Neural Machine Translation of Rare Words with Subword Units*

Corpus:

```
low</w>
low</w>
lower</w>
lowest</w>
high</w>
high</w>
higher</w>
```

Segmentation:

Merges:

```
w </w>  (2)
r </w>  (2)
h </w>  (2)
t </w>  (1)
```

given this set of merges, let's segment the corpus!

Sennrich et al. (2016): *Neural Machine Translation of Rare Words with Subword Units*

Corpus:	Segmentation:	Merges:
low</w>	l o w</w>	w </w> (2)
low</w>	l o w</w>	r </w> (2)
lower</w>	l o w e r</w>	h </w> (2)
lowest</w>	l o w e s t</w>	t </w> (1)
high</w>	h i g h</w>	
high</w>	h i g h</w>	
higher</w>	h i g h e r</w>	

Sennrich et al. (2016): *Neural Machine Translation of Rare Words with Subword Units*

Corpus:	Segmentation:
low</w>	l o w</w>
low</w>	l o w</w>
lower</w>	l o w e r</w>
lowest</w>	l o w e s t</w>
high</w>	h i g h</w>
high</w>	h i g h</w>
higher</w>	h i g h e r</w>

Merges:

```
w </w> (2)
r </w> (2)
h </w> (2)
t </w> (1)
```

looking at the segmented corpus shows us what merge will occur next

Corpus:	Segmentation:	Merges:
low</w>	l o w</w>	w </w> (2)
low</w>	l o w</w>	r </w> (2)
lower</w>	l o w e r</w>	h </w> (2)
lowest</w>	l o w e s t</w>	t </w> (1)
high</w>	h i g h</w>	
high</w>	h i g h</w>	
higher</w>	h i g h e r</w>	

what is the next merge?
(what unit bigram appears most often?)

Sennri… …re
Words …

Corpus:	Segmentation:	Merges:
low</w>	l o w</w>	w </w> (2)
low</w>	l o w</w>	r </w> (2)
lower</w>	l o w e r</w>	h </w> (2)
lowest</w>	l o w e s t</w>	t </w> (1)
high</w>	h i g h</w>	l o (4)
high</w>	h i g h</w>	
higher</w>	h i g h e r</w>	

given this new set of merges, let's re-segment the corpus!

Sennrich et al. (2
Words with Subw

Corpus:	Segmentation:
low</w>	lo w</w>
low</w>	lo w</w>
lower</w>	lo w e r</w>
lowest</w>	lo w e s t</w>
high</w>	h i g h</w>
high</w>	h i g h</w>
higher</w>	h i g h e r</w>

Merges:

```
w </w>  (2)
r </w>  (2)
h </w>  (2)
t </w>  (1)
l o     (4)
```

Sennrich et al. (2016): *Neural M*
Words with Subword Units

note: we will always "back off" to the complete segmentation

Corpus:	Segmentation:	Merges:
low</w>	lo w</w>	w </w> (2)
low</w>	lo w</w>	r </w> (2)
lower</w>	lo w e r</w>	h </w> (2)
lowest</w>	lo w e s t</w>	t </w> (1)
high</w>	h i g h</w>	l o (4)
high</w>	h i g h</w>	
higher</w>	h i g h e r</w>	

what is the next merge?
(what unit bigram appears most often?)

Sennrich et al., 2016, *Neural Machine Translation of Rare Words with Subword Units*

Corpus:	Segmentation:	Merges:
low</w>	lo w</w>	w </w> (2)
low</w>	lo w</w>	r </w> (2)
lower</w>	lo w e r</w>	h </w> (2)
lowest</w>	lo w e s t</w>	t </w> (1)
high</w>	h i g h</w>	l o (4)
high</w>	h i g h</w>	i g (3)
higher</w>	h i g h e r</w>	

Corpus:	Segmentation:	Merges:
low</w>	lo w</w>	w </w> (2)
low</w>	lo w</w>	r </w> (2)
lower</w>	lo w e r</w>	h </w> (2)
lowest</w>	lo w e s t</w>	t </w> (1)
high</w>	h ig h</w>	l o (4)
high</w>	h ig h</w>	i g (3)
higher</w>	h ig h e r</w>	

Corpus:	Segmentation:	Merges:
low</w>	lo w</w>	w </w> (2)
low</w>	lo w</w>	r </w> (2)
lower</w>	lo w e r</w>	h </w> (2)
lowest</w>	lo w e s t</w>	t </w> (1)
high</w>	h ig h</w>	l o (4)
high</w>	h ig h</w>	i g (3)
higher</w>	h ig h e r</w>	h ig (3)

Corpus:	Segmentation:	Merges:
low</w>	lo w</w>	w </w> (2)
low</w>	lo w</w>	r </w> (2)
lower</w>	lo w e r</w>	h </w> (2)
lowest</w>	lo w e s t</w>	t </w> (1)
high</w>	hig h</w>	l o (4)
high</w>	hig h</w>	i g (3)
higher</w>	hig h e r</w>	h ig (3)

Corpus:	Segmentation:
low</w>	low</w>
low</w>	low</w>
lower</w>	lowe r</w>
lowest</w>	lowe s t</w>
high</w>	high</w>
high</w>	high</w>
higher</w>	hig h e r</w>

Merges:

```
w </w>  (2)
r </w>  (2)
h </w>  (2)
t </w>  (1)
l o     (4)
i g     (3)
h ig    (3)
w e     (2)
lo we   (2)
lo w</w>(2)
hig h</w>(2)
```

we'd like to merge `e` and `r</w>`, but we merged `w` and `e` already, so that messed us up

New Corpus:

```
low (2x)
lower
lowest
high (2x)
higher
small
smaller
smallest
```

Merges for New Corpus:

```
l o
s m
sm a
sma l
i g
h ig
e r</w>
smal l
s t</w>
lo w</w>
lo w
hig h</w>
e st</w>
```

Merges:

```
l o
s m
sm a
sma l
i g
h ig
e r</w>
smal l
s t</w>
lo w</w>
lo w
hig h</w>
e st</w>
```

Application:

low → low

lower → low er

lowest → low est

high → high

higher → hig h er

highest → hig h est

small → smal l

smaller → small er

smallest → small est

- we can limit the vocabulary size of the segmented data by limiting the number of merges

- this can be very helpful for handling an open vocabulary of words while reducing computation (e.g., when using a softmax over the vocabulary)

Language Modeling

Background

- Language Model?
  - Predict what word comes next



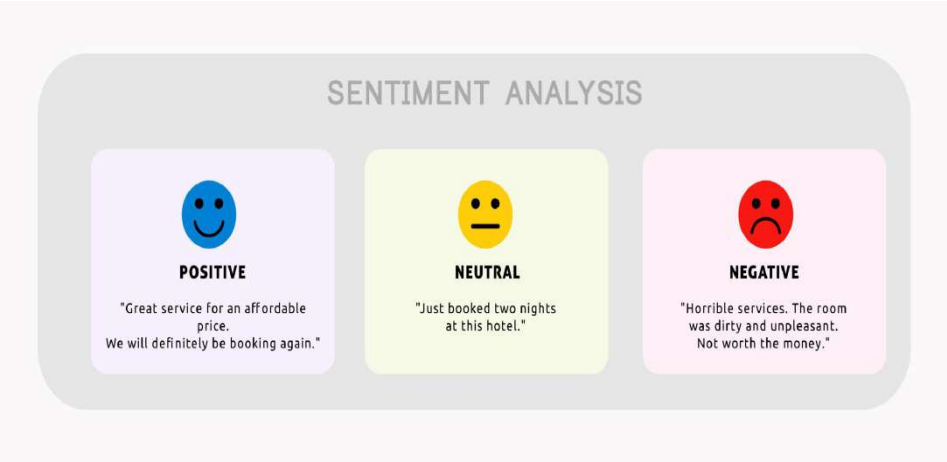
Start to learn English

Learn to start business

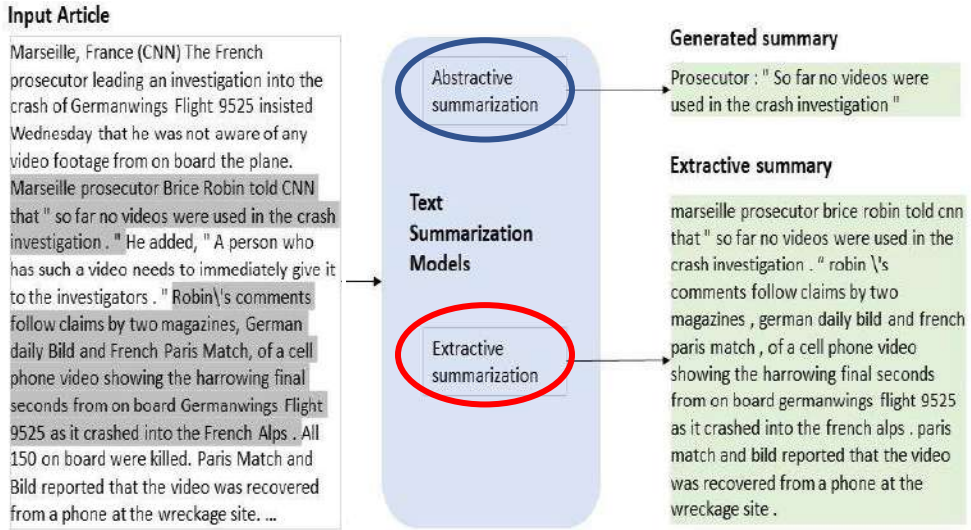
Word order matters!

Language models are everywhere

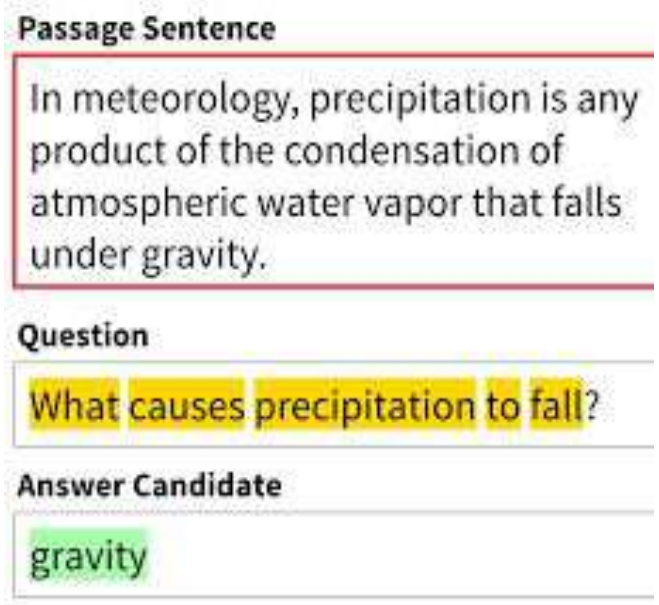
Sentiment



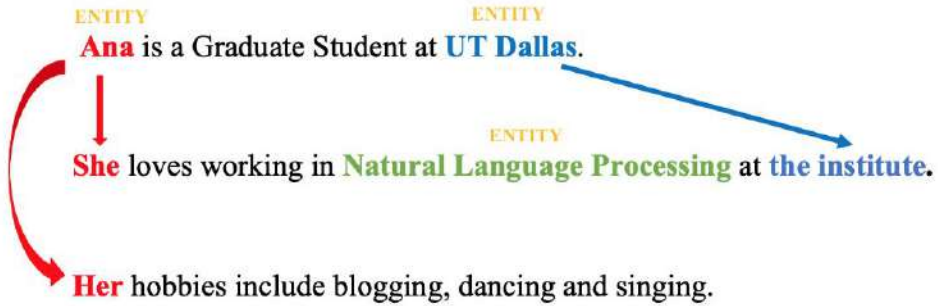
Summarization



Question Answering



Coreference Resolution



Language models are everywhere

General Language Understanding Evaluation



- Popular GLUE NLP tasks
  - Paraphrase detection (MRPC: Microsoft Research Paraphrase Corpus)
  - Natural language inference
  - Named entity recognition
  - Shallow syntax
  - Word sense disambiguation
  - Textual entailment
  - The Corpus of linguistic acceptability

https://gluebenchmark.com/leaderboard

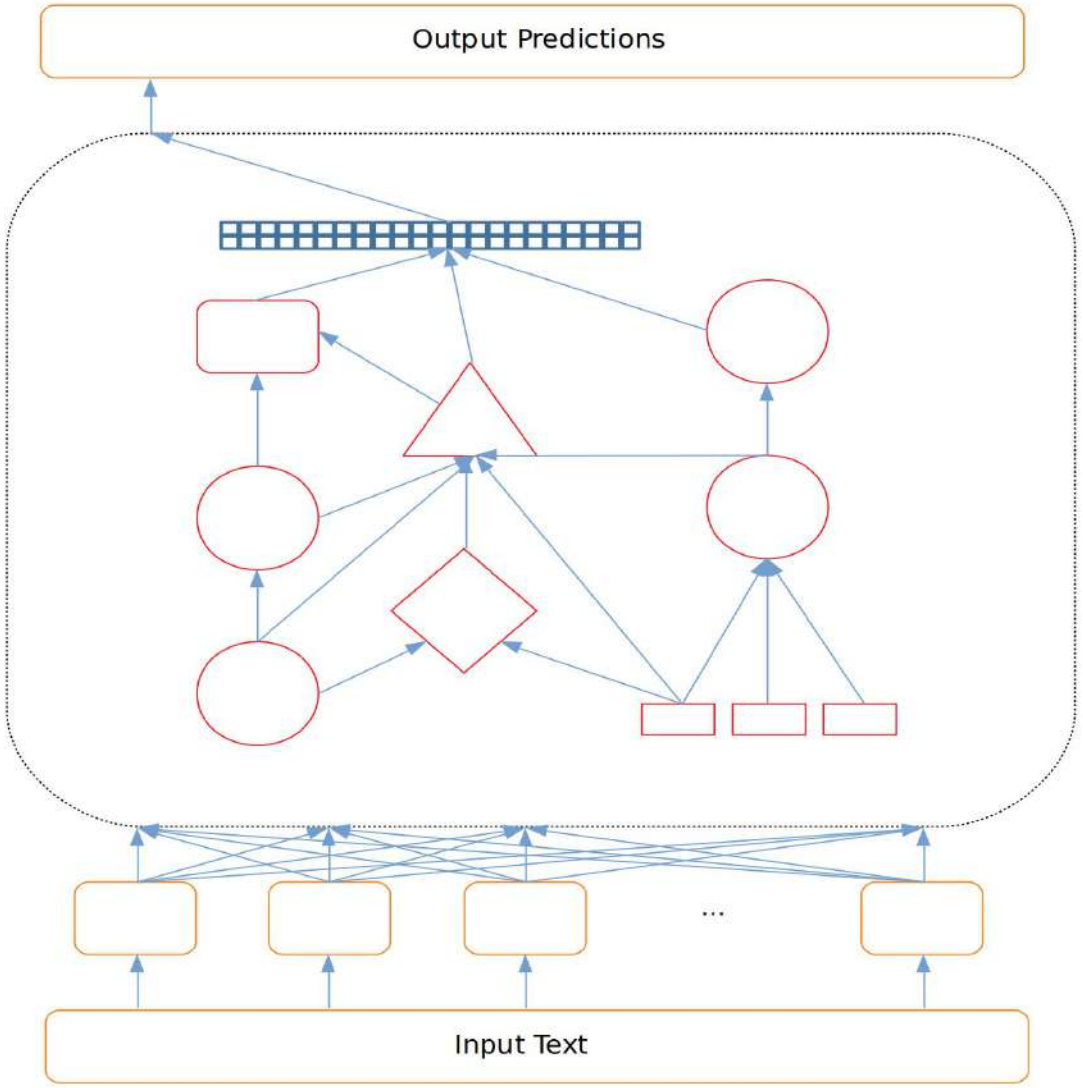
GLUE Tasks

Name	Metric
The Corpus of Linguistic Acceptability	Matthew's Corr
The Stanford Sentiment Treebank	Accuracy
Microsoft Research Paraphrase Corpus	F1 / Accuracy
Semantic Textual Similarity Benchmark	Pearson-Spearman Corr
Quora Question Pairs	F1 / Accuracy
MultiNLI Matched	Accuracy
MultiNLI Mismatched	Accuracy
Question NLI	Accuracy
Recognizing Textual Entailment	Accuracy
Winograd NLI	Accuracy
Diagnostics Main	Matthew's Corr

Dataset	Description	Data example	Metric
CoLA	Is the sentence grammatical or ungrammatical?	"This building is than that one." = **Ungrammatical**	Matthews
SST-2	Is the movie review positive, negative, or neutral?	"The movie is funny , smart , visually inventive , and most of all , alive ." = **.93056 (Very Positive)**	Accuracy
MRPC	Is the sentence B a paraphrase of sentence A?	A) "Yesterday , Taiwan reported 35 new infections , bringing the total number of cases to 418 ." B) "The island reported another 35 probable cases yesterday , taking its total to 418 ." = **A Paraphrase**	Accuracy / F1
STS-B	How similar are sentences A and B?	A) "Elephants are walking down a trail." B) "A herd of elephants are walking along a trail." = **4.6 (Very Similar)**	Pearson / Spearman
QQP	Are the two questions similar?	A) "How can I increase the speed of my internet connection while using a VPN?" B) "How can Internet speed be increased by hacking through DNS?" = **Not Similar**	Accuracy / F1
MNLI-mm	Does sentence A entail or contradict sentence B?	A) "Tourist Information offices can be very helpful." B) "Tourist Information offices are never of any help." = **Contradiction**	Accuracy
QNLI	Does sentence B contain the answer to the question in sentence A?	A) "What is essential for the mating of the elements that create radio waves?" B) "Antennas are required by any radio receiver or transmitter to couple its electrical connection to the electromagnetic field." = **Answerable**	Accuracy
RTE	Does sentence A entail sentence B?	A) "In 2003, Yunus brought the microcredit revolution to the streets of Bangladesh to support more than 50,000 beggars, whom the Grameen Bank respectfully calls Struggling Members." B) "Yunus supported more than 50,000 Struggling Members." = **Entailed**	Accuracy
WNLI	Sentence B replaces sentence A's ambiguous pronoun with one of the nouns - is this the correct noun?	A) "Lily spoke to Donna, breaking her concentration." B) "Lily spoke to Donna, breaking Lily's concentration." = **Incorrect Referent**	Accuracy

https://gluebenchmark.com/leaderboard

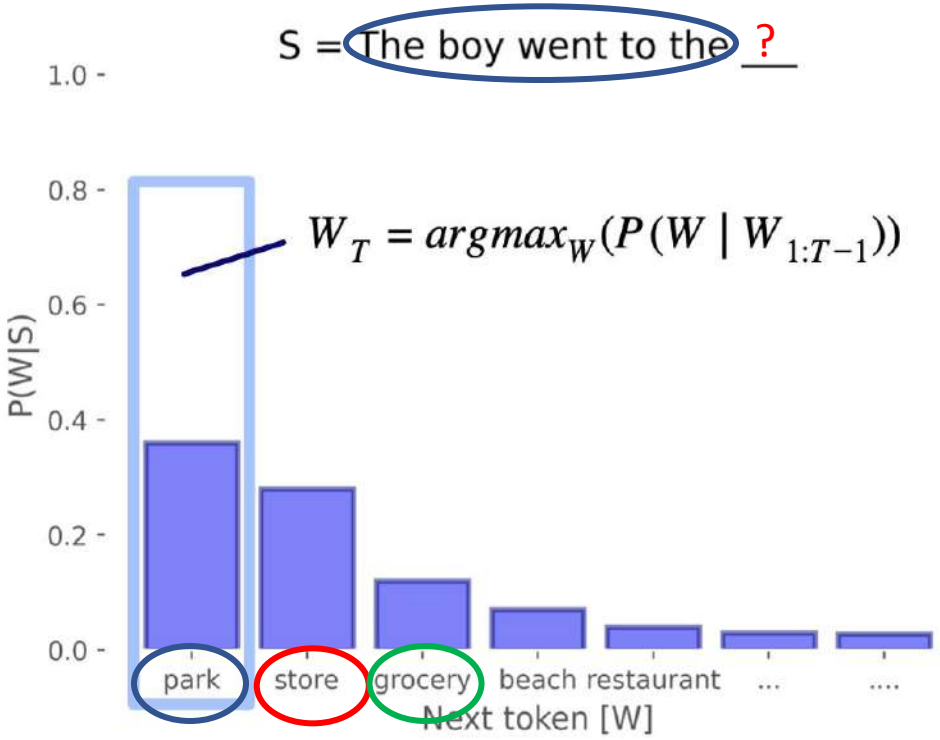
GLUE Architecture



LSTMs,
RNNs
Word2Vec
GloVe
ELMo
Transformers (BERT, GPT-2, etc,.)

Statistical Language Models

- The goal of statistical language modelling is to calculate the probability distribution over sequence of words.



once upon a time	0.0000614344%
once upon a long	0.0000000786%
once upon a day	0.0000001017%
once upon a handsomely	0.0000000878%
once upon a dream	0.0000000832%
once upon a real	0.0000000508%
once upon a very	0.0000000231%
once upon a times	0.0000000739%
once upon a successful	0.0000000231%
once upon a lifetime	0.0000000231%

Statistical Language Models

- The goal of statistical language modelling is to calculate the probability distribution over sequence of words.

- Unigram model

$$P(w_1 w_2 \dots w_n) \approx \prod_i P(w_1)$$

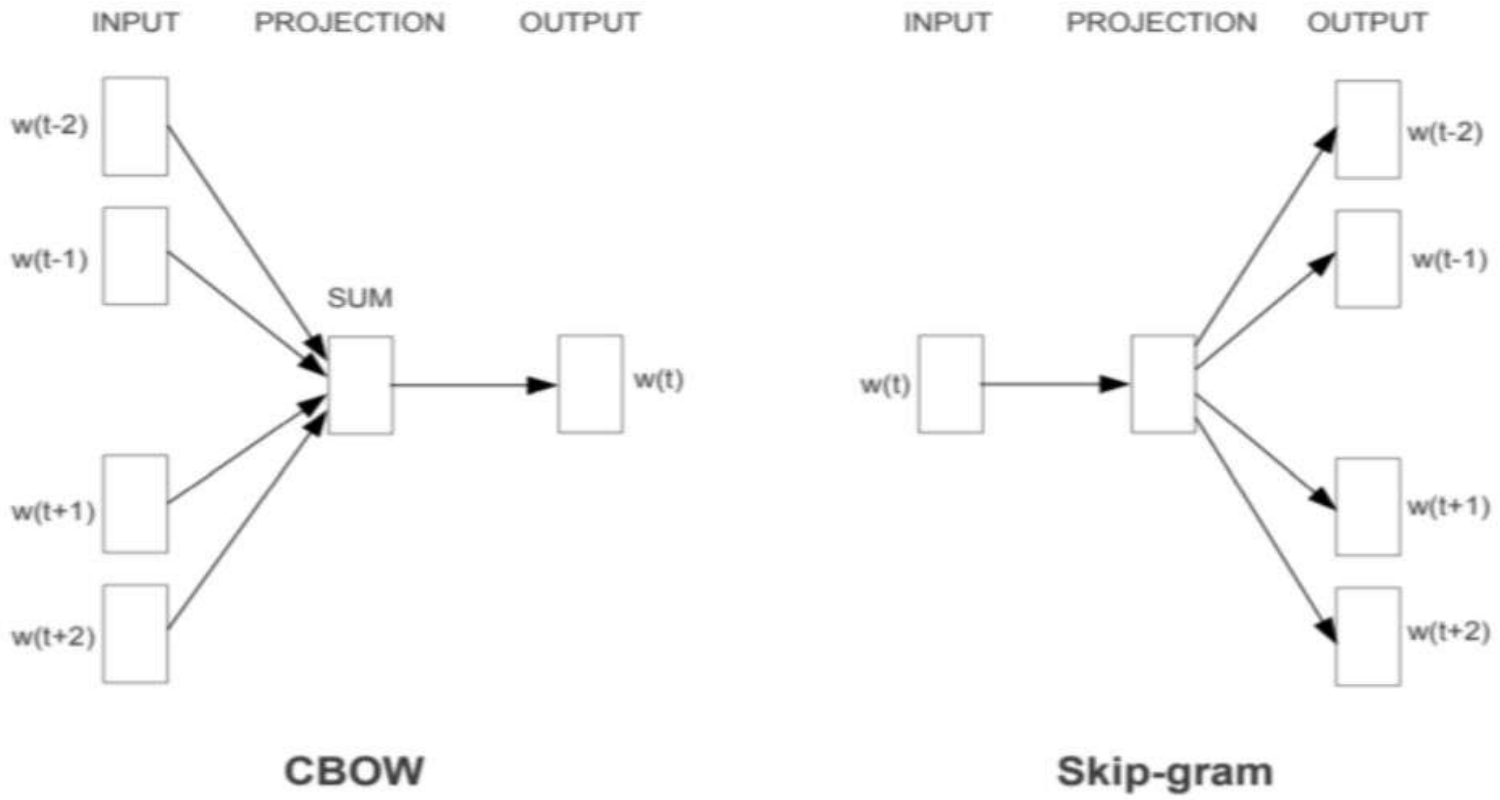
- N-gram model

$$P(w_i | w_1 w_2 \dots w_{i-1}) \approx P(w_i | w_{i-(n-1)} \dots w_{i-1})$$

$$P(w_i | w_{i-1}) = \frac{count(w_{i-1}, w_i)}{count(w_{i-1})}$$

Neural Language Models

- Word2Vec
  - Learns distributed representations of words.
  - Continuous bag-of-words (CBOW): Predicts current word from a window of surrounding context words.
  - Continuous skip-gram: Uses current word to predict surrounding window of context words.
- Similar models such as GloVe, FastText



https://papers.nips.cc/paper/2013/file/9aa42b31882ec039965f3c4923ce901b-Paper.pdf

Probabilistic language models

Goal: compute the probability of a sentence (or sequence of words):

$$P(\mathbf{w}) = P(w_1, w_2, w_3, \ldots, w_n)$$

Related task: probability of the next word:

$$P(w_5 \mid w_4, w_3, w_2, w_1)$$

A model that computes either of these is called a **language model** (or **LM**).

How do we compute the joint probability:

$$P(w_1, w_2, w_3, \ldots, w_n) = \ ?$$

Language modelling = assigning probabilities to words in a sentence

Which sequences of words are likely to appear together?

Probabilistic language models

Goal: compute the probability of a sentence (or sequence of words):

$$P(\mathbf{w}) = P(w_1, w_2, w_3, \ldots, w_n)$$

Related task: probability of the next word:

$$P(w_5 \mid w_4, w_3, w_2, w_1)$$

A model that computes either of these is called a **language model** (or **LM**).

Want to compute the joint probability:

$$P(w_1, w_2, w_3, \ldots, w_n) = \prod_i P(w_i \mid w_1, w_2, \ldots, w_{i-1})$$

For example:

$$P(its, water, is, so, transparent, that) = P(its) \cdot P(water \mid its) \cdot P(is \mid its\ water) \cdot P(so \mid its\ water\ is) \cdot P(transparent \mid its\ water\ is\ so)$$

Language modelling = assigning probabilities to words in a sentence

Which sequences of words are likely to appear together?

Markov assumption

Make the simplifying assumption:

$$P(\textit{the} \mid \textit{its water is so transparent that}) \approx P(\textit{the} \mid \textit{that})$$

Or maybe:

$$P(\textit{the} \mid \textit{its water is so transparent that}) \approx P(\textit{the} \mid \textit{transparent that})$$

$$P(w_1, w_2, w_3, \ldots, w_n) \approx \prod_i P(w_i \mid w_{i-1}, \ldots, w_{i-k})$$

In other words, approximate each component of the product:

$$P(w_i \mid w_{i-1}, \ldots, w_2, w_1) \approx P(w_i \mid w_{i-1}, \ldots, w_{i-k})$$



Andrei Markov
1856 — 1922

How many words?

Bigrams, 3-grams, 4-grams, 5-grams, …

Not perfect (not capturing long term dependencies):

*The **computer(s)** that I just put*
*into the machine room on the fifth floor **is (are)** crashing.*

n-grams require accounting for V^n events. $V=10^4$, $n=7 \rightarrow 10^{28}$ events





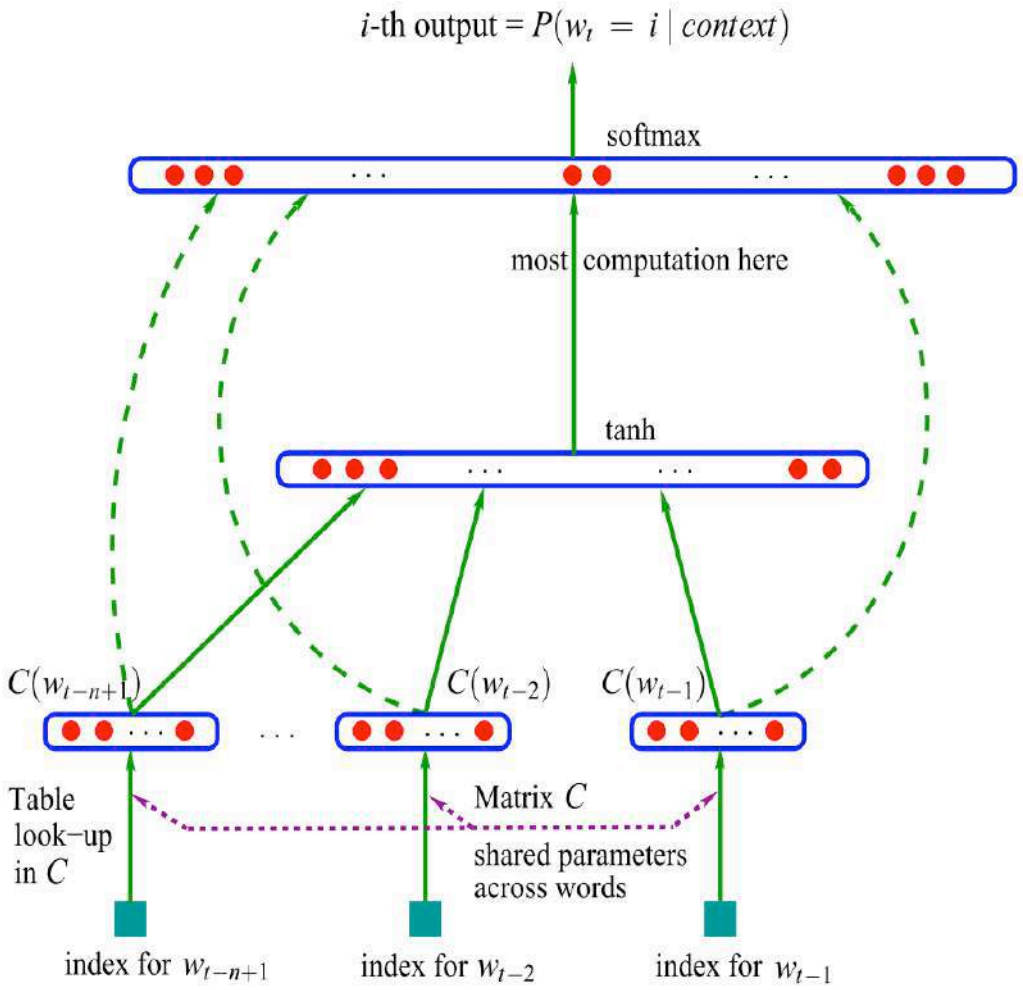
Neural Language Models

Inspired by the lectures of Julia Tsvetkov (CMU), Jacob Eisenstein (Georgia Tech) and David Bamman (Berkeley)

A History Lesson

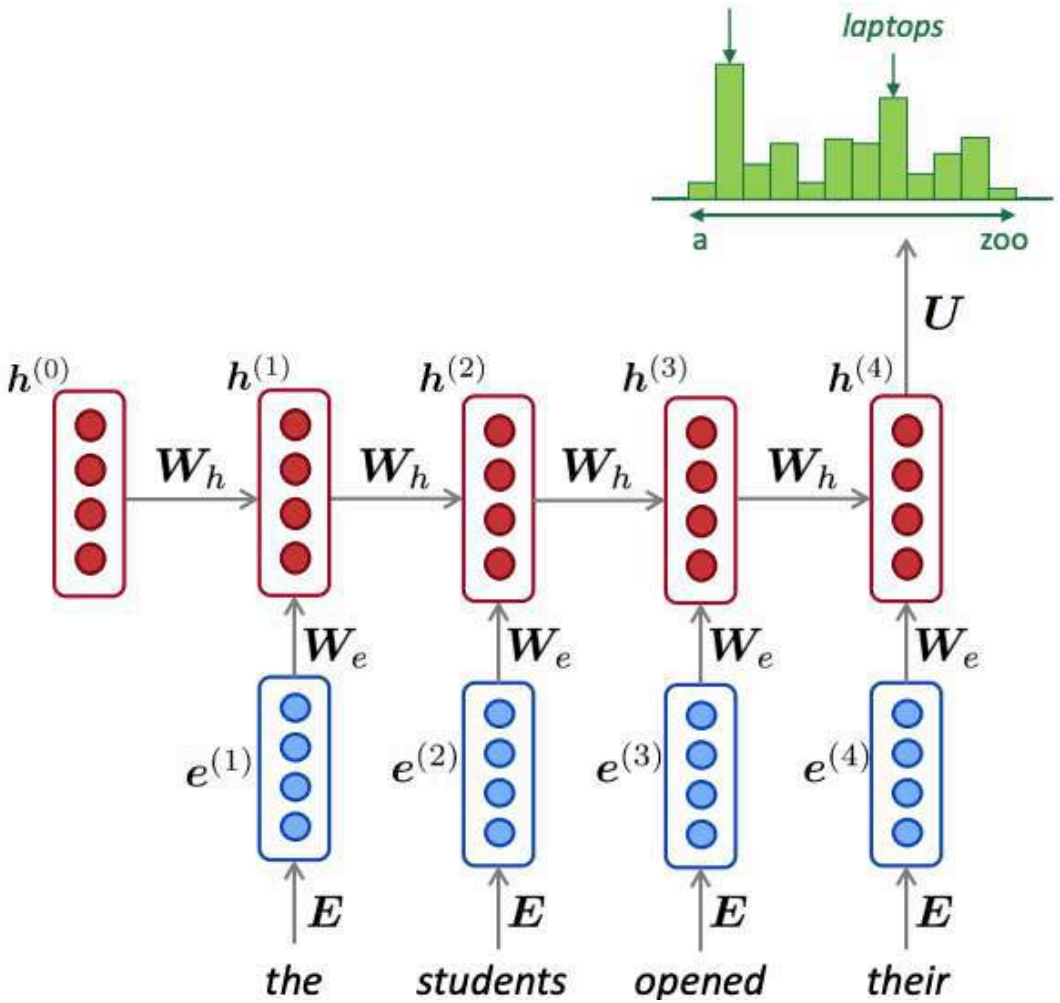
Feed-forward Neural Network Language Model

2001



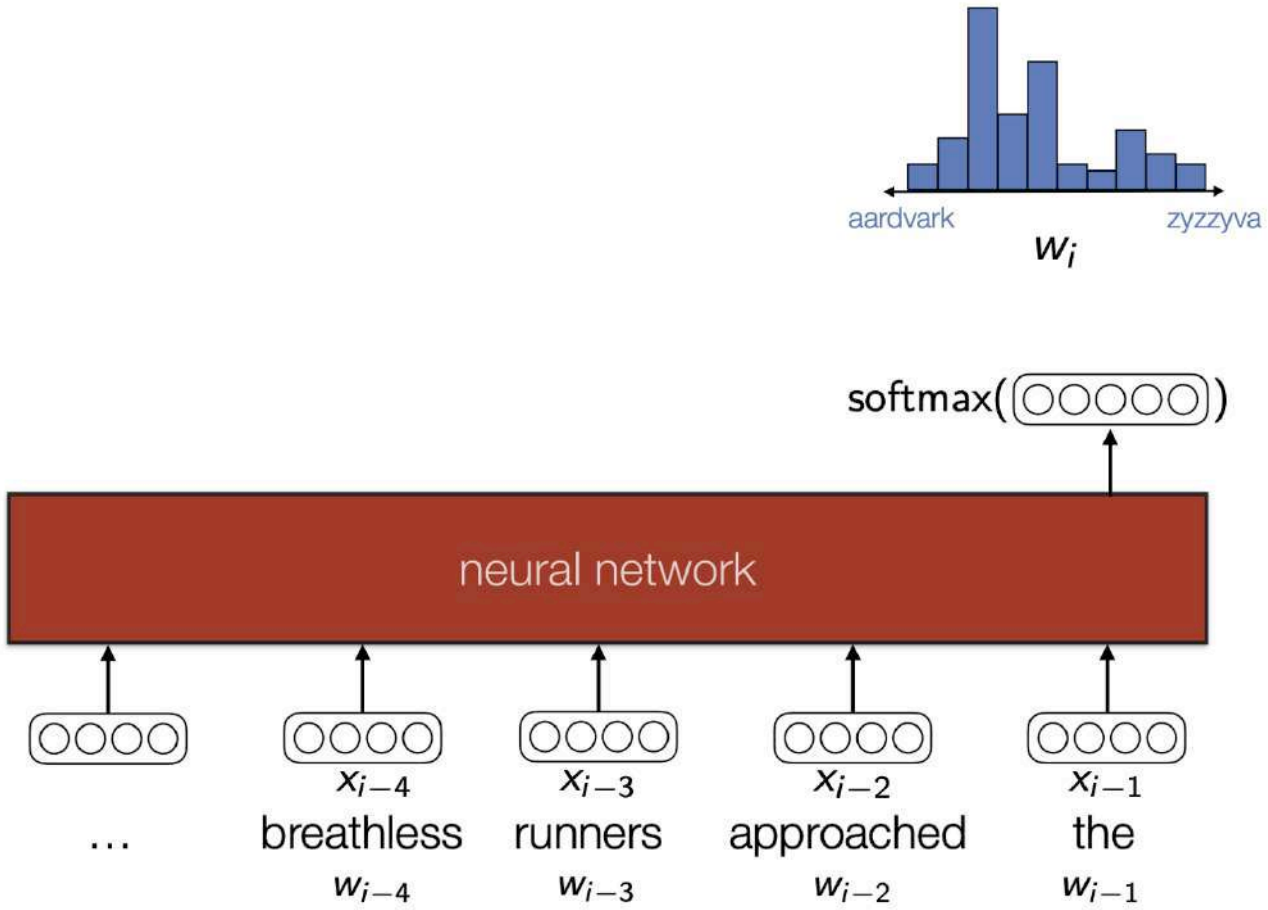
[1] Bengio, Y., Ducharme, R., & Vincent, P. (2001). A Neural Probabilistic Language Model. Proceedings of NIPS.

Neural Language Models



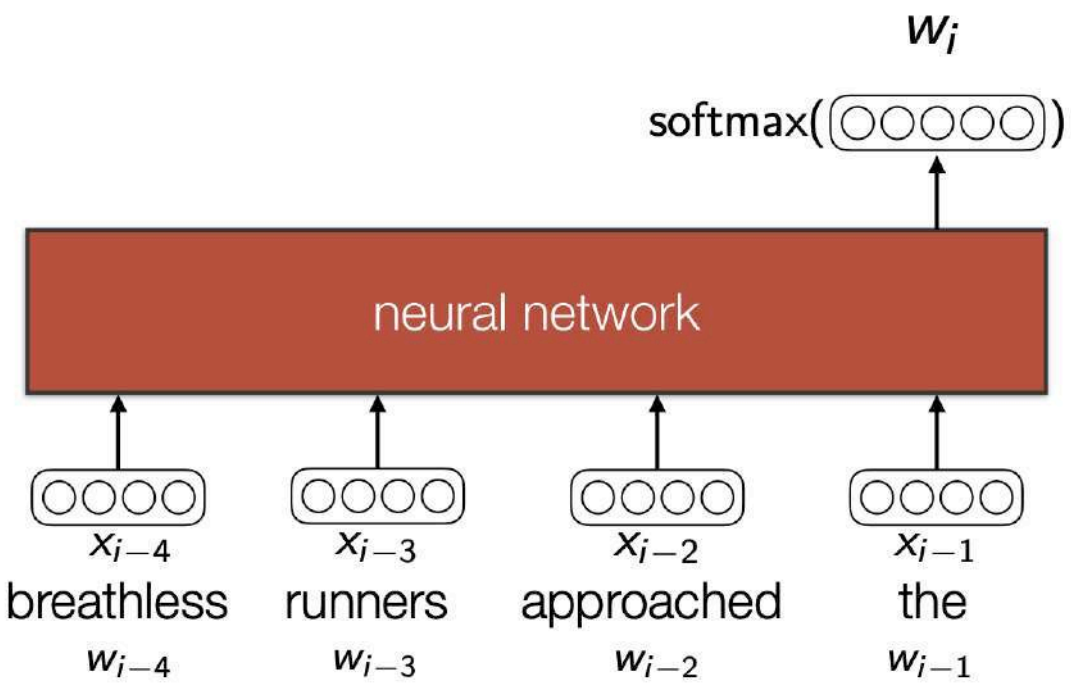
- Better empirical performance at language modelling (lower perplexity, better generalization, longer context)
- Provide state-of-the-art dense word representations (used instead of word embeddings these days)
- But: a lot more computation. Slower to train and test

Neural Language Models



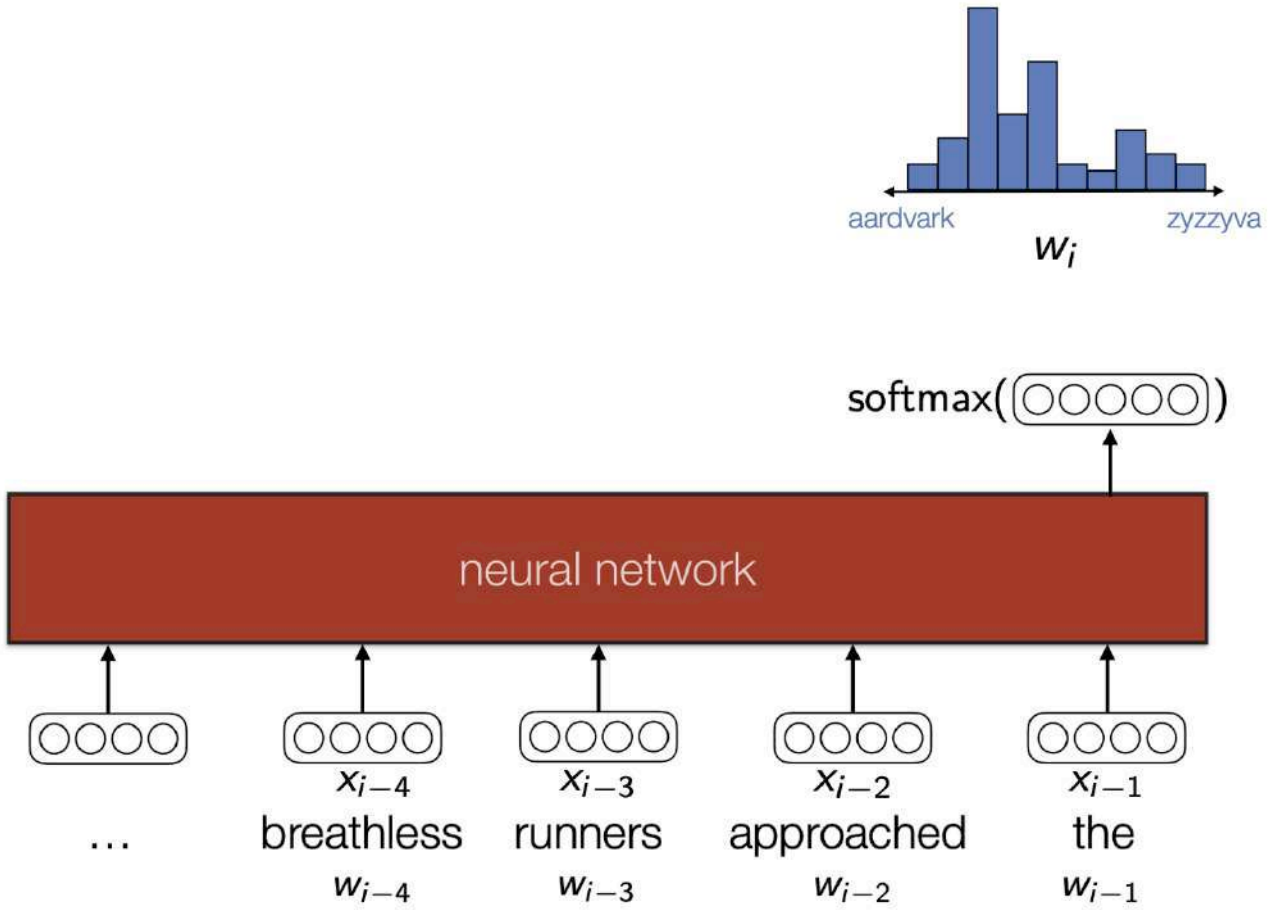
- Idea: predicting probabilities rather than counting
- Input:
  word embeddings
  $(x_1, x_2, \ldots, x_N)$
- Output:
  $P(w_i \mid w_{i-1}, w_{i-2}, \ldots, w_2, w_1)$

Option 1: Feed-forward neural LM



- Let's fix the window size (e.g. k = 4 words)
- Input: concatenated k word embeddings into one long vector
  $x = [x_{i-4}, x_{i-3}, x_{i-2}, x_{i-1}]$

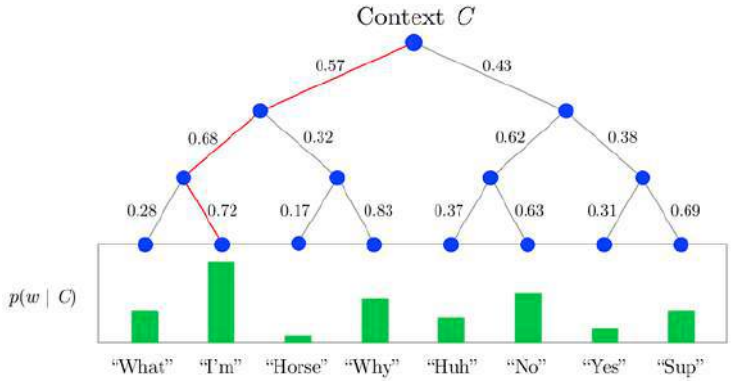
Softmax over large vocabulary



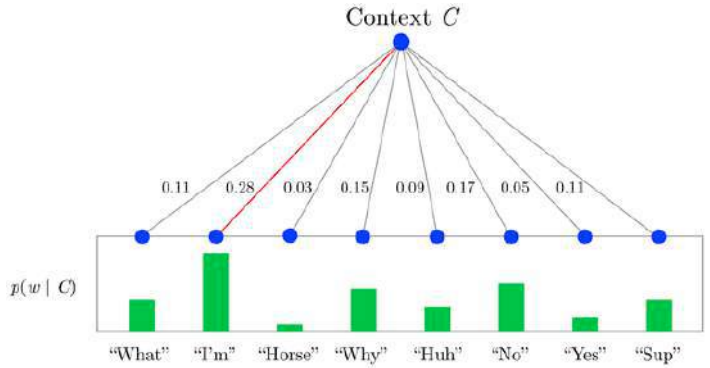
- inconvenient to score the next word over entire vocabulary (similar issues as in the n-gram model)
- Some solutions:
  - Hierarchical softmax
  - Noise-contrastive estimation (NCE)
  - Adaptive softmax

Softmax over large vocabulary

Hierarchical Softmax uses a multi-layer binary tree, where the probability of a word is calculated through the product of probabilities on each edge on the path to that node:



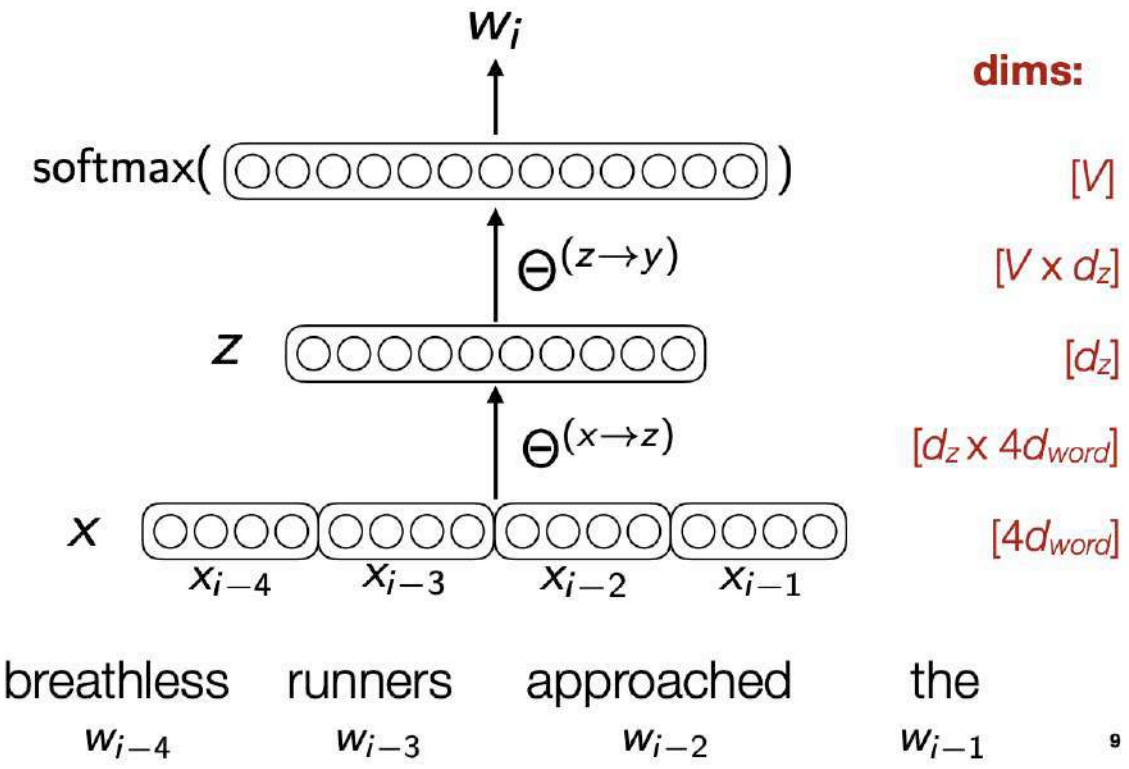
Instead of:



- inconvenient to score the next word over entire vocabulary (similar issues as in the n-gram model)
- Some solutions:
  - **Hierarchical softmax**
  - Noise-contrastive estimation (NCE)
  - Adaptive softmax

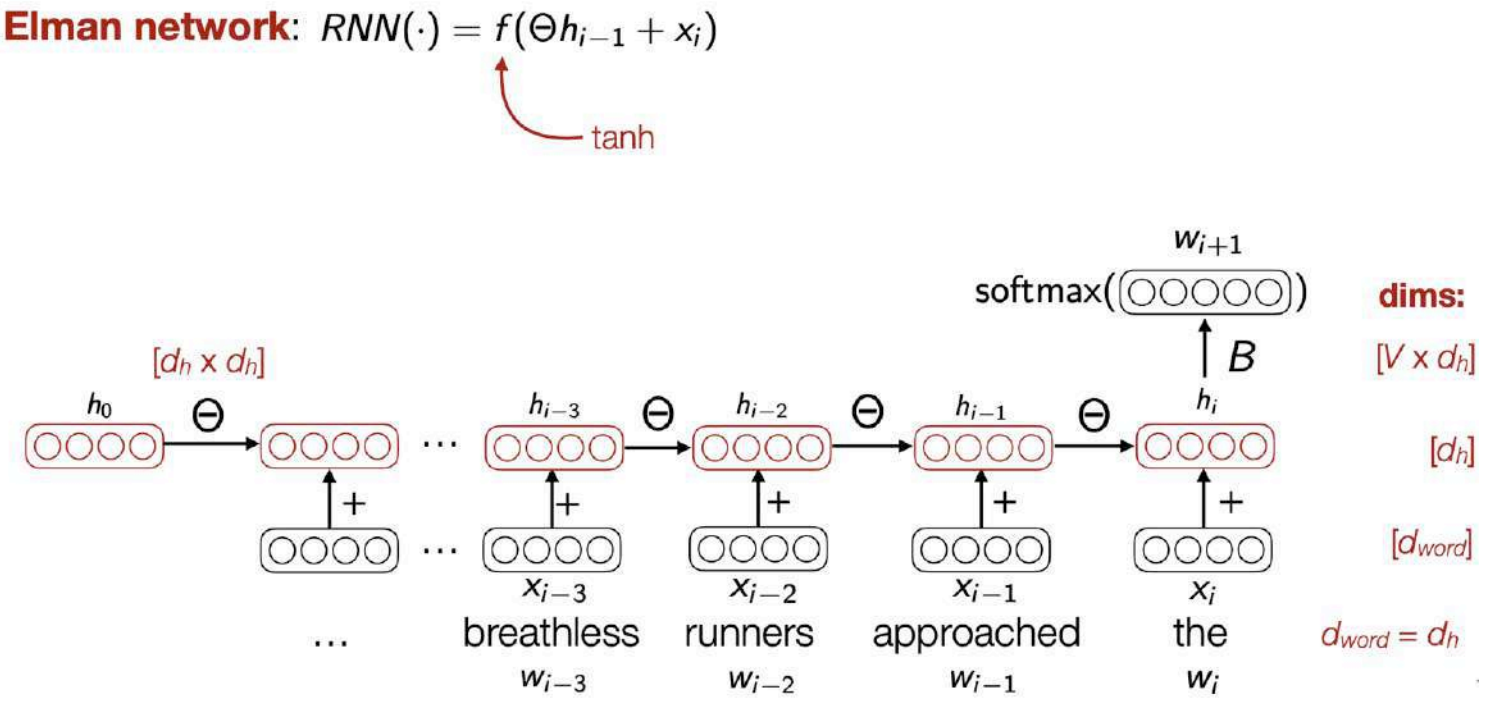
Option 1: Feed-forward neural LM

$$P(w_i = j \mid z) = \frac{\exp(\theta_j^{(z \to y)} \cdot z)}{\sum_{j' \in V} \exp(\theta_{j'}^{(z \to y)} \cdot z)}$$



- Let's fix the window size (e.g. k = 4 words)
- Input: concatenated k word embeddings into one long vector $x = [x_{i-4}, x_{i-3}, x_{i-2}, x_{i-1}]$
- Fully connected hidden layer $f(\Theta^{(x \to z)} x)$
- Output: $\text{softmax}(\Theta^{(z \to y)} z)$

Option 2: Recurrent neural LM

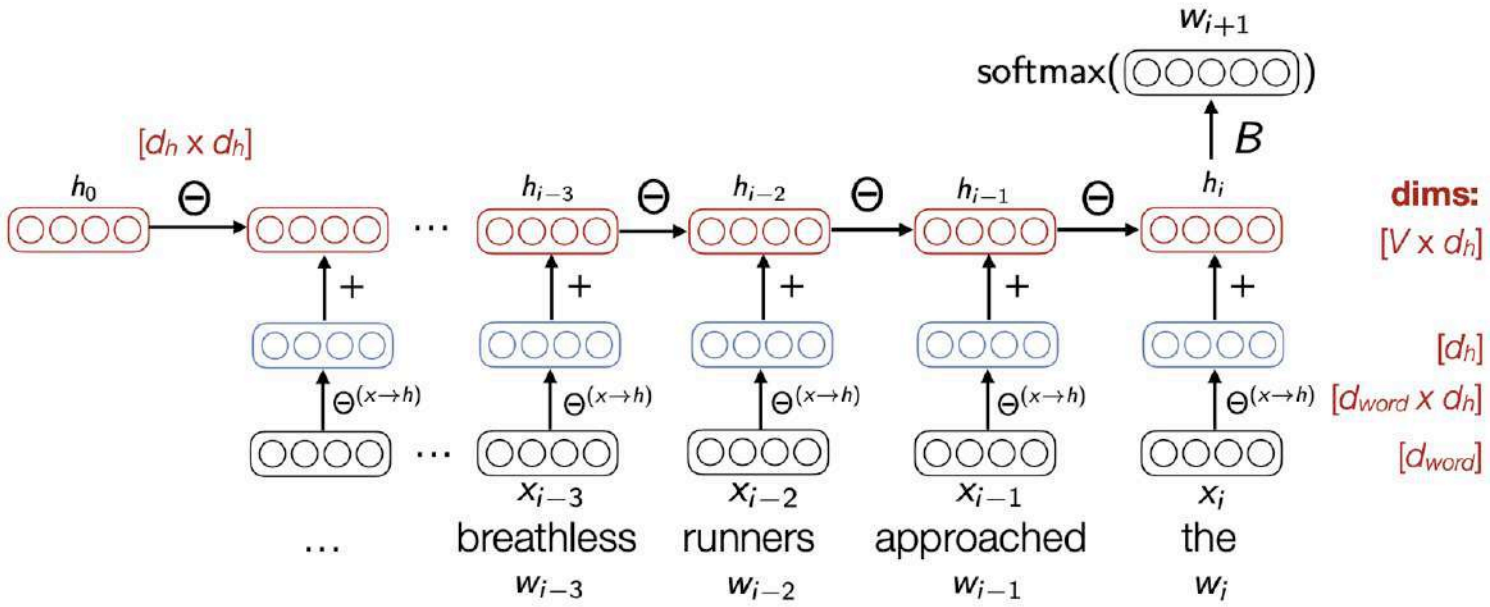


- Maintain a context vector h
- At each timestep (w_j), compose the context with the current word x_j to create a new context for the next timestep:

$$h_j = RNN(x_j, h_{j-1})$$

- Recurrent because we repeatedly apply the same function RNN(·) each time to consider previous step
- If $d_{word} = d_h$, we can tie input (X) and output (B) word embeddings

Option 2: Recurrent neural LM



- Maintain a context vector h
- At each timestep (w_j), compose the context with the current word x_j to create a new context for the next timestep:

$$h_j = \text{RNN}(x_j, h_{j-1})$$

- Recurrent because we repeatedly apply the same function RNN(·) each time to consider previous step
- If we don't want to tie X with B, we can also add another projection matrix $\Theta^{(x \to h)}$

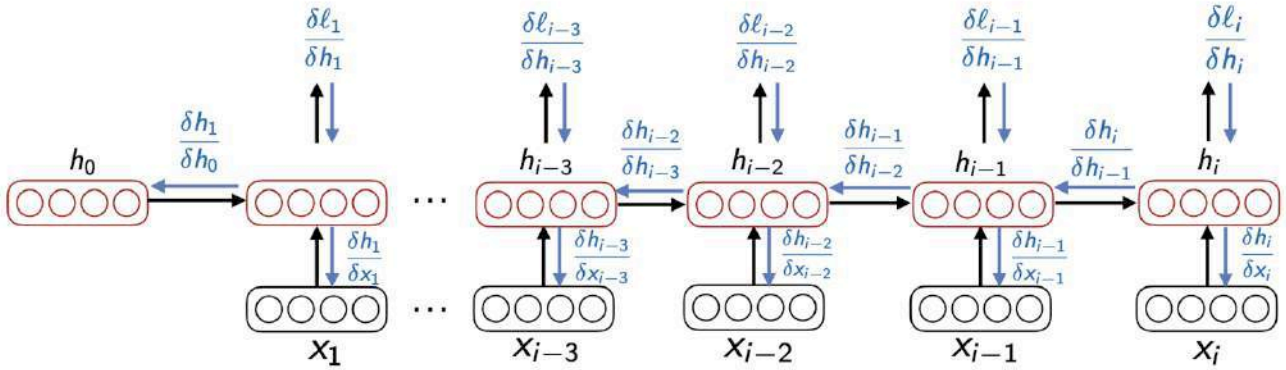
Option 2: Recurrent neural LM

Learning recurrent neural LMs: Backprop through time

In addition to each layer, now gradients for the stochastic gradient descent must be computed with respect to each timestep as well.
"Unroll" the RNN and treat the parameters at each timestep as if it were another layer; apply the chain rule.

(for details, see the NLP Lecture 3 or the corresponding chapter in Jacob Eisenstein's book)

Scan me to Learn more about BPTT

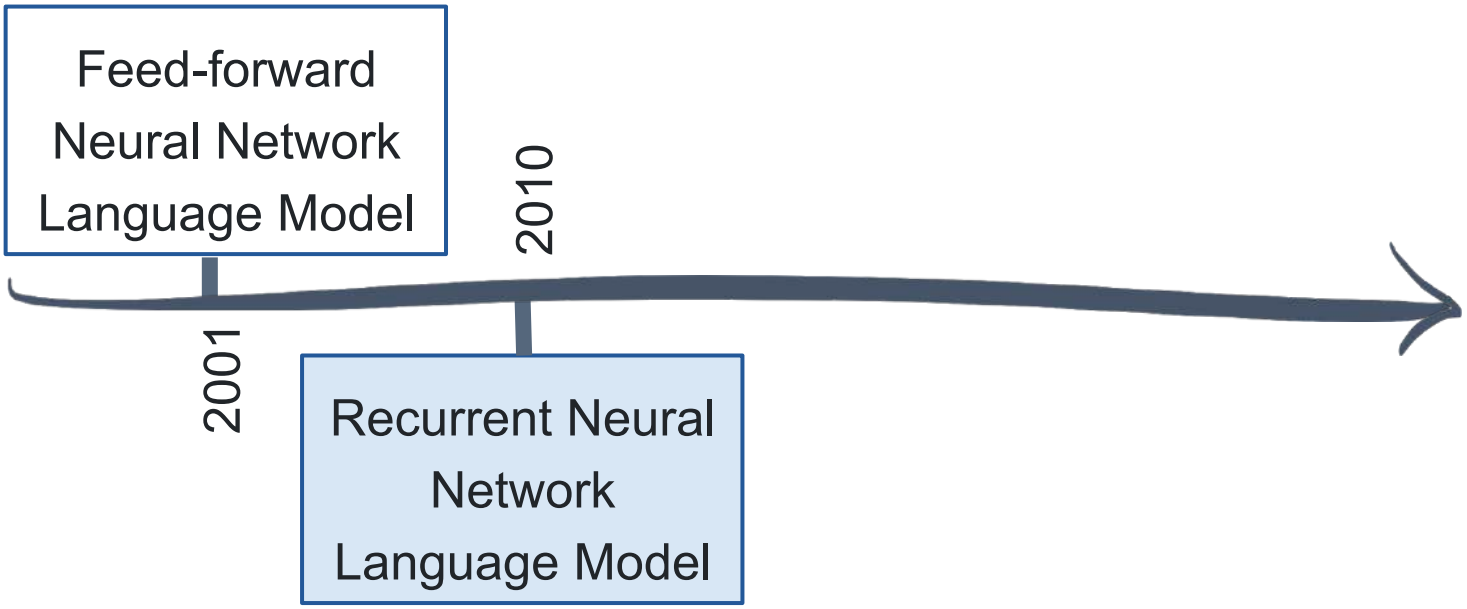


- Maintain a context vector h
- At each timestep (w_j), compose the context with the current word x_j to create a new context for the next timestep:

$$h_j = \text{RNN}(x_j, h_{j-1})$$

- Recurrent because we repeatedly apply the same function RNN(·) each time to consider previous step

A History Lesson



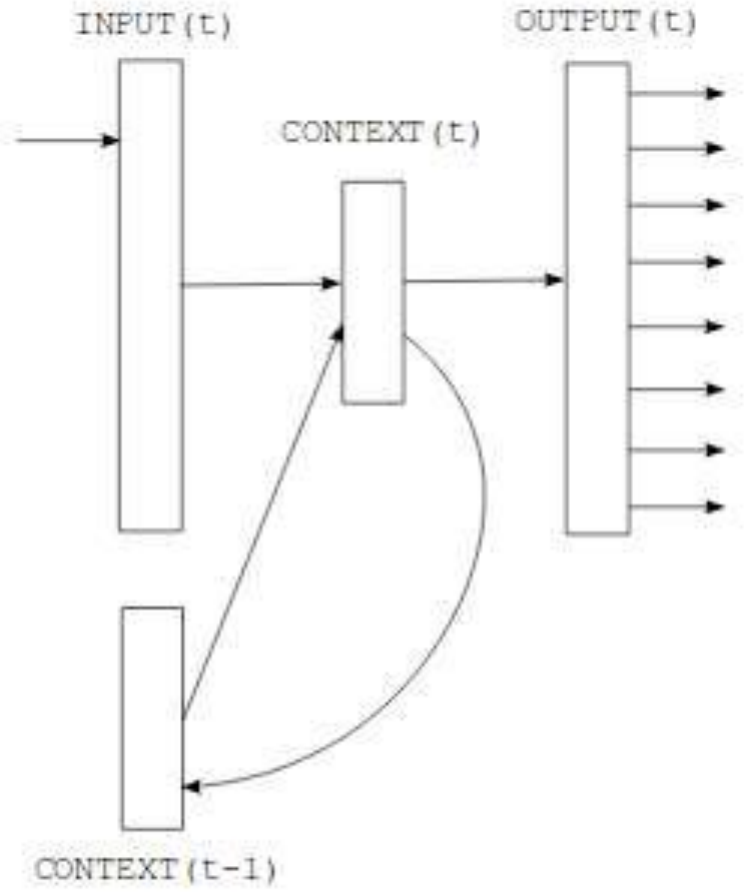


Figure 1: *Simple recurrent neural network.*

Elman, J. L. (1990). Finding structure in time. Cognitive science, 14(2), 179-211.

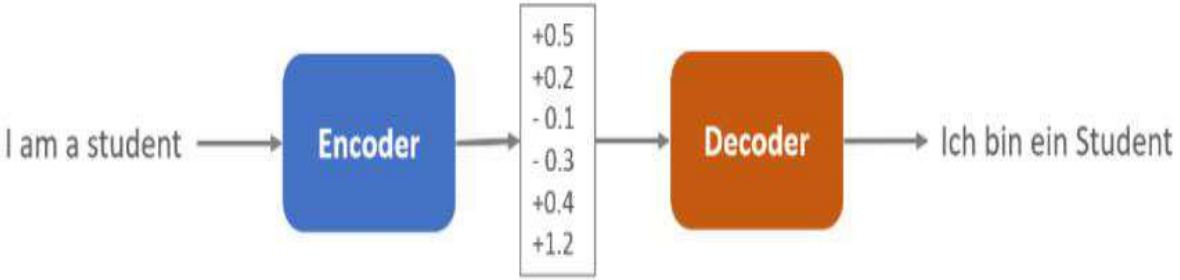
Mikolov, T., Karafiát, M., Burget, L., Černocký, J., & Khudanpur, S. (2010). Recurrent neural network based language model. In Eleventh Annual Conference of the International Speech Communication Association.

Why do we need RNNs?

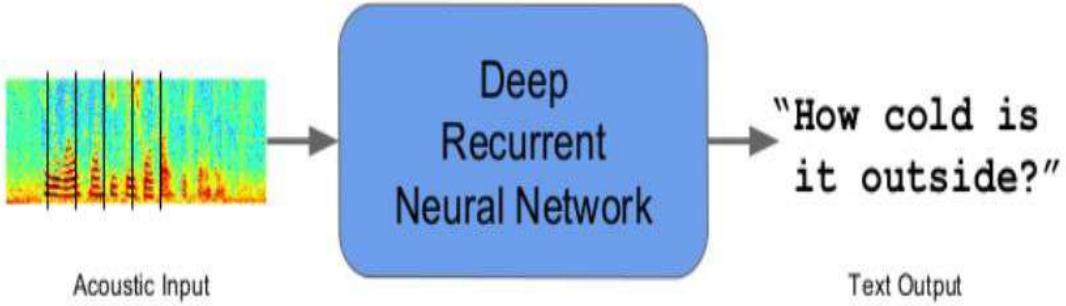
- What is Sequence Learning?
- Ways to process sequences using RNNs.

What is Sequence Learning?

Machine Translation



Speech Recognition



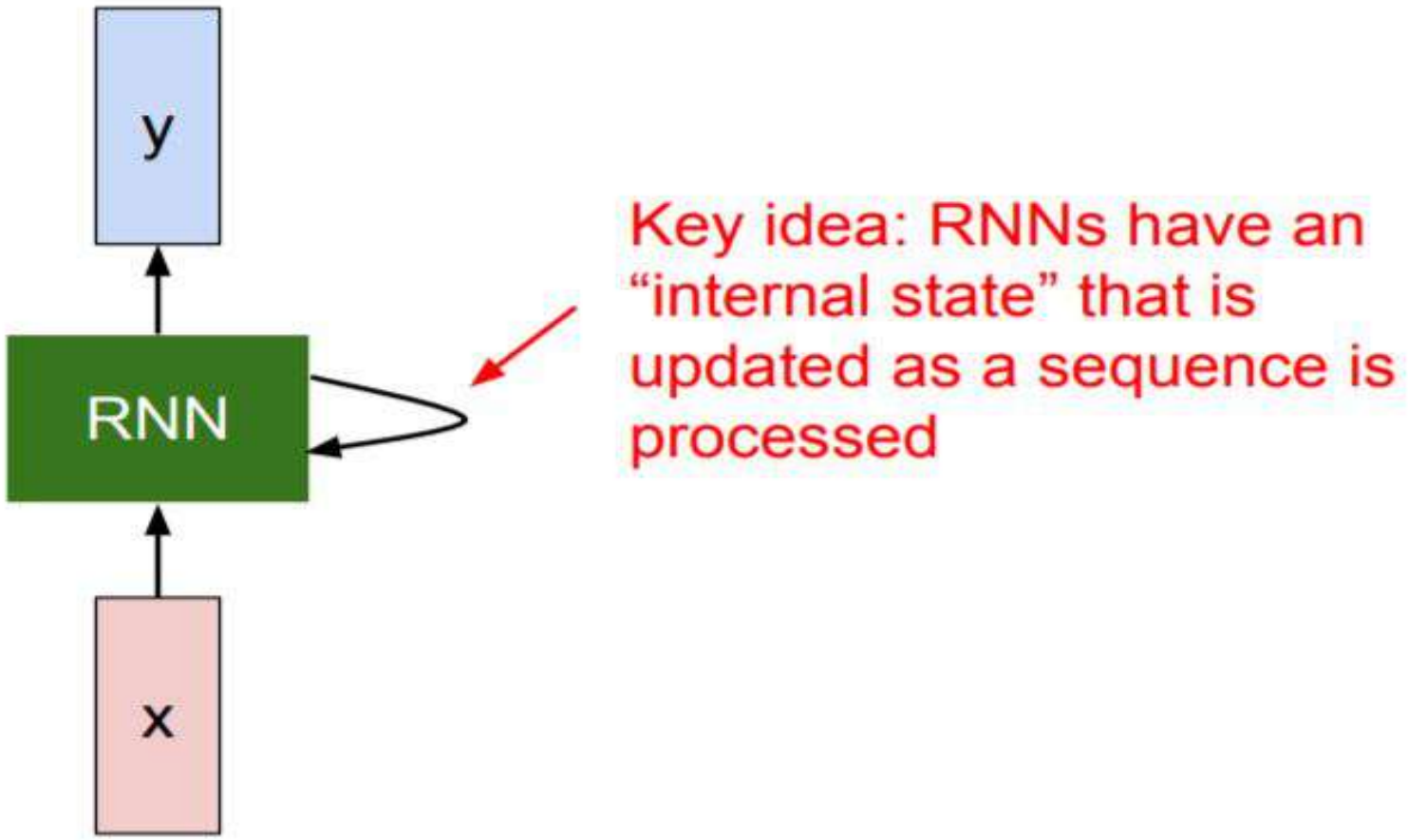
Named Entity Recognition

| | | |

Different Recurrent Methods?

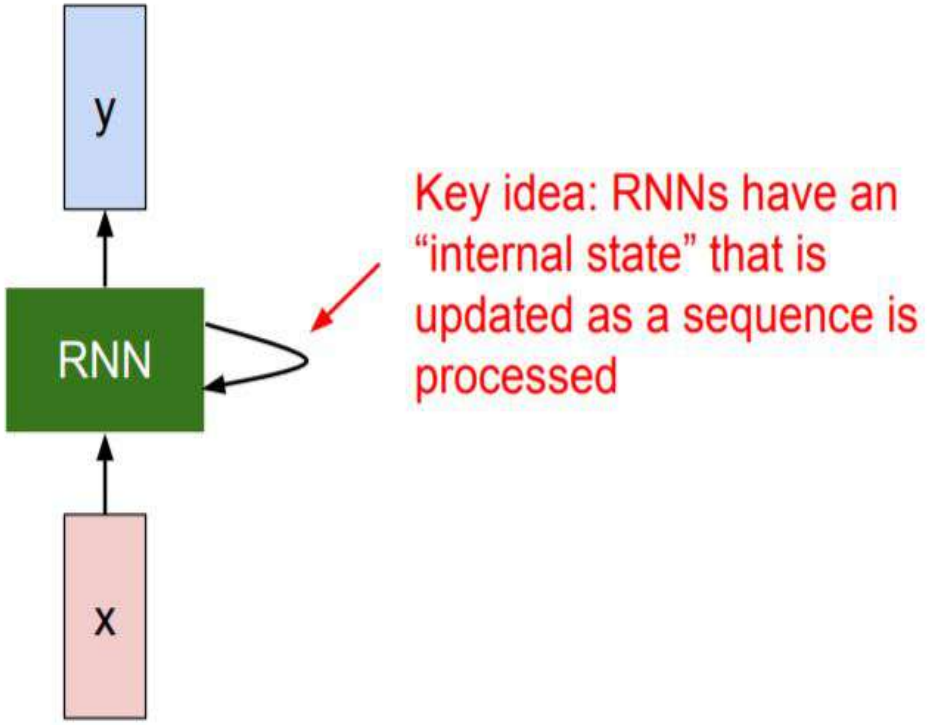
- Echo State Networks (i.e Reservoir Computing) (ESN)
- Recurrent Neural Networks (RNN)
- Long Short-Term Memory Networks (LSTM)
- Gated Recurrent Units (GRU)

RNNs



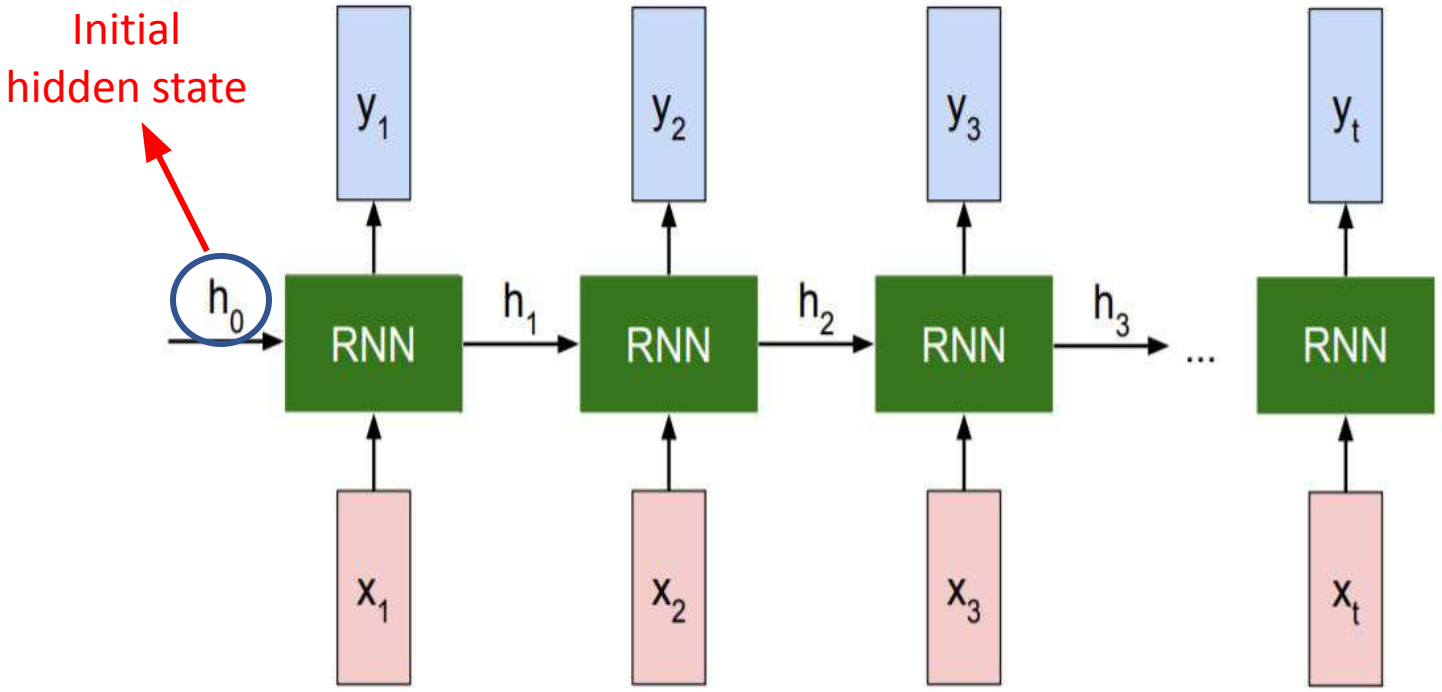
Source: Fei-Fei Li slide:19

RNNs



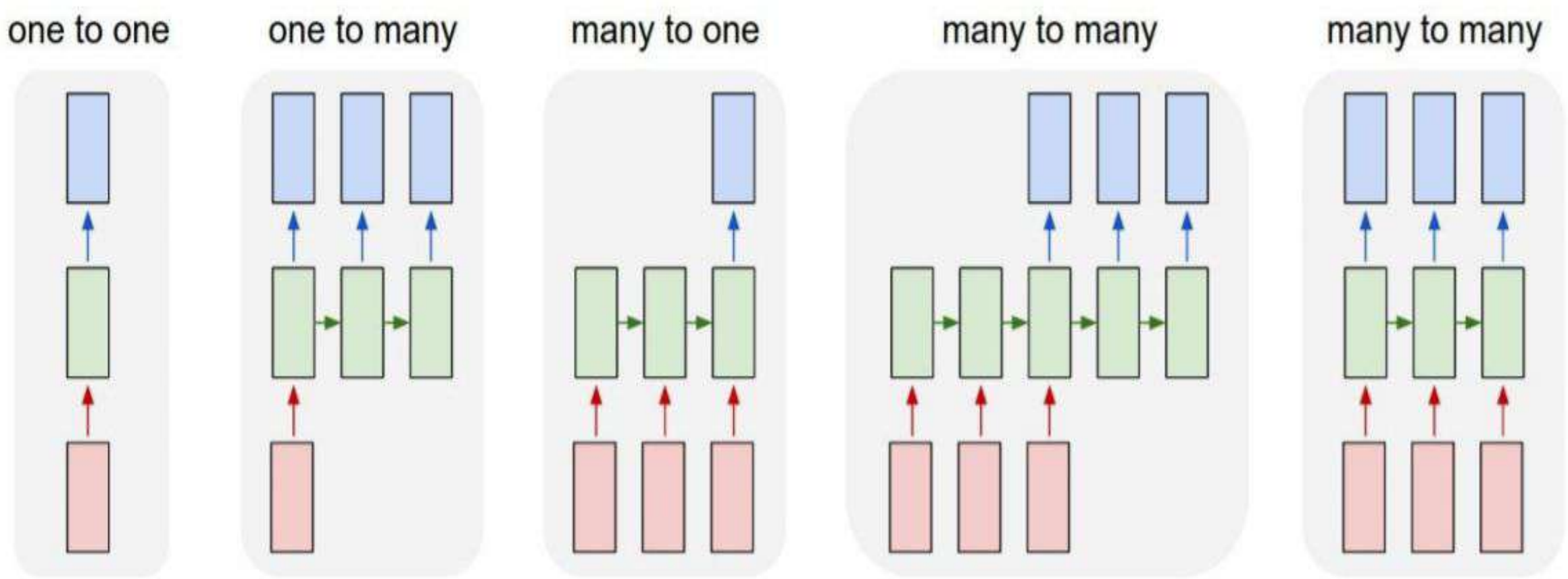
Key idea: RNNs have an "internal state" that is updated as a sequence is processed

Rolled RNN



Unrolled RNN

Ways to Process Sequences using RNNs



Vanilla Neural Networks

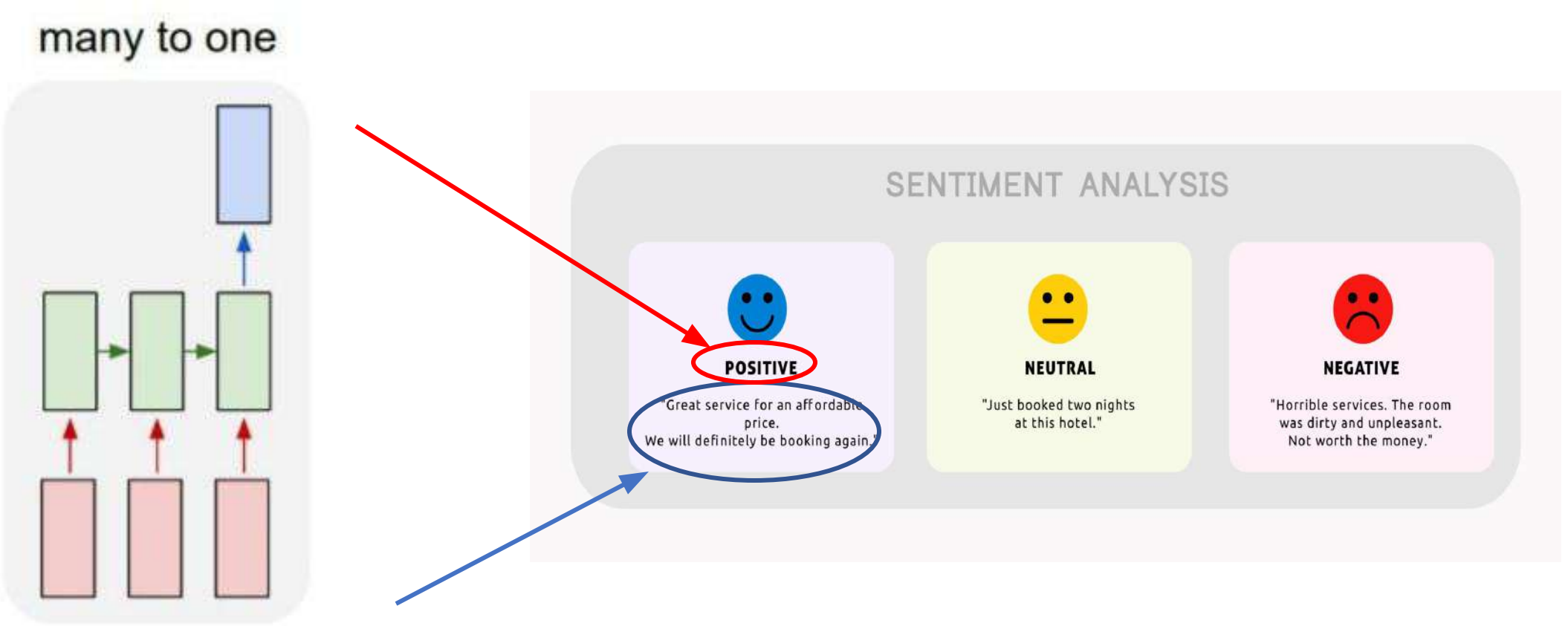
Image Captioning

action prediction

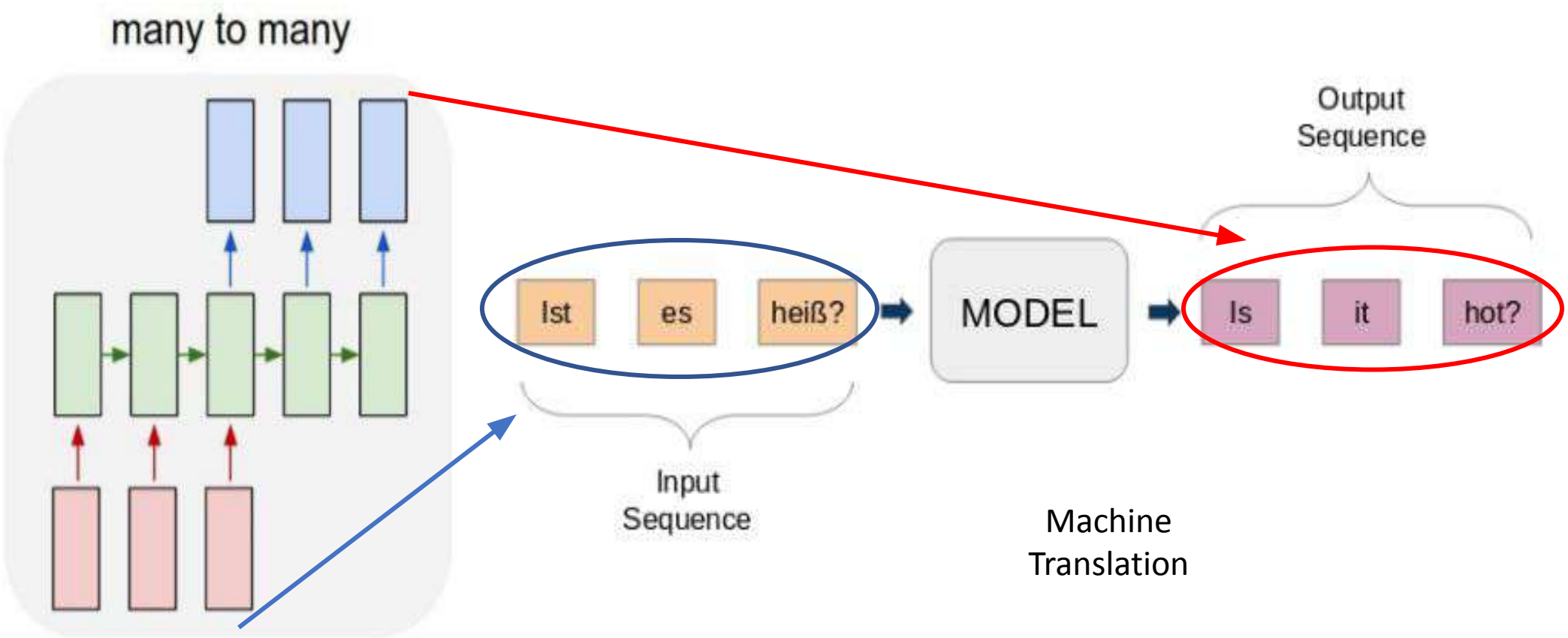
Video Captioning

Video classification

Many-to-One


many to one
SENTIMENT ANALYSIS
POSITIVE
"Great service for an affordable price.
We will definitely be booking again."
NEUTRAL
"Just booked two nights at this hotel."
NEGATIVE
"Horrible services. The room was dirty and unpleasant. Not worth the money."

Many-to-Many


many to many
Ist
es
heiß?
MODEL
Is
it
hot?
Output Sequence
Input Sequence
Machine Translation

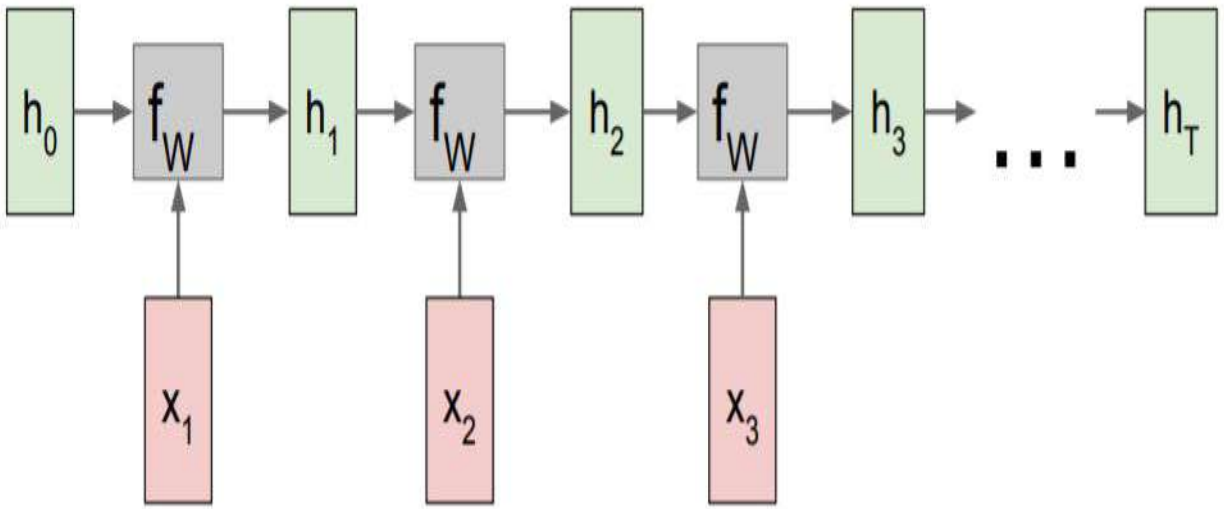
RNN Computational Graph

$$h_t = f_W(h_{t-1}, x_t)$$

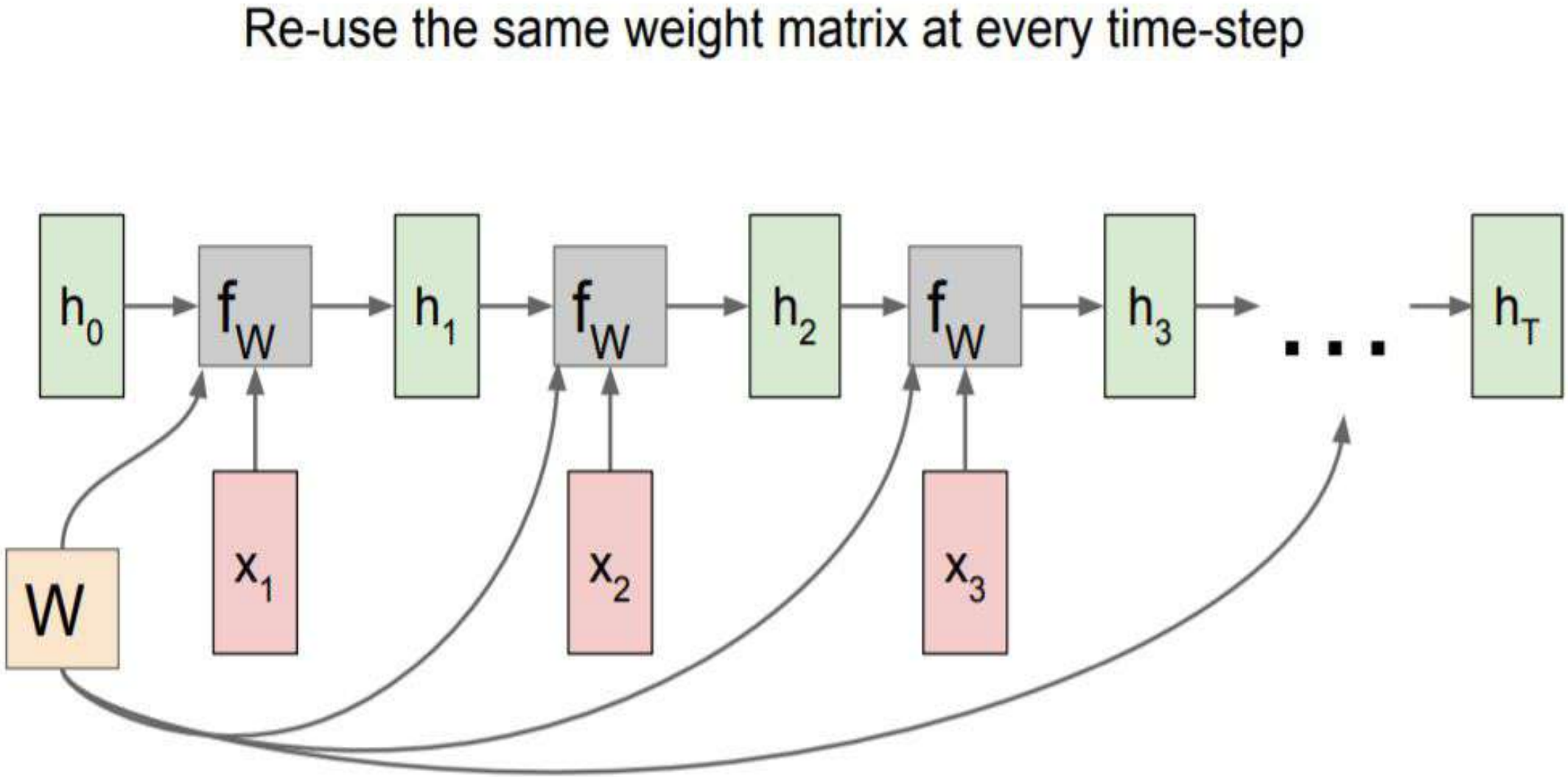
$$\downarrow$$

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$

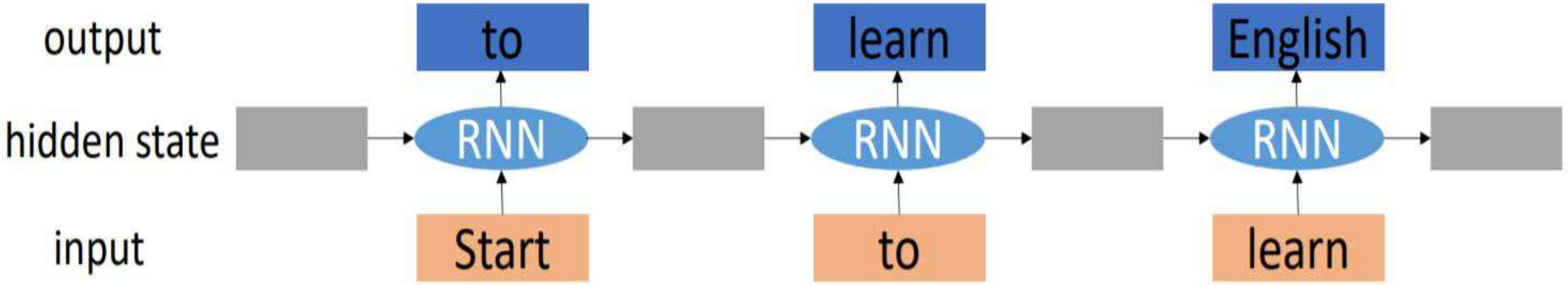


RNN: Computational Graph

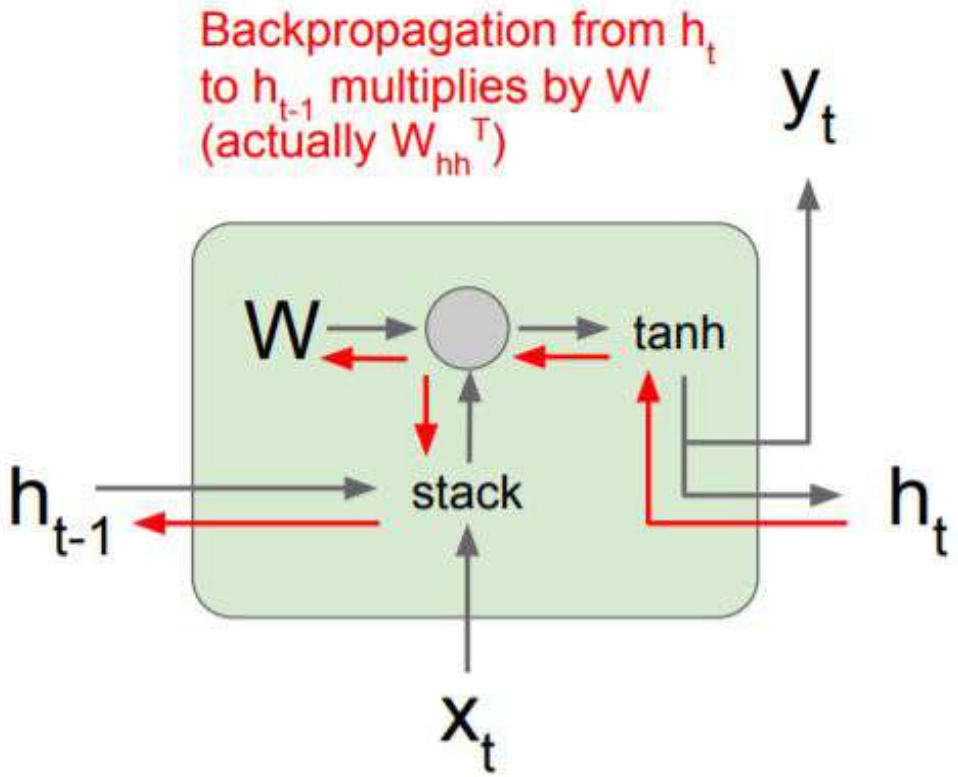


Language Models with RNNs

- RNN models the word order in hidden state
- Natural fit for sentences and sequences of pixels.



RNN Gradient Flow

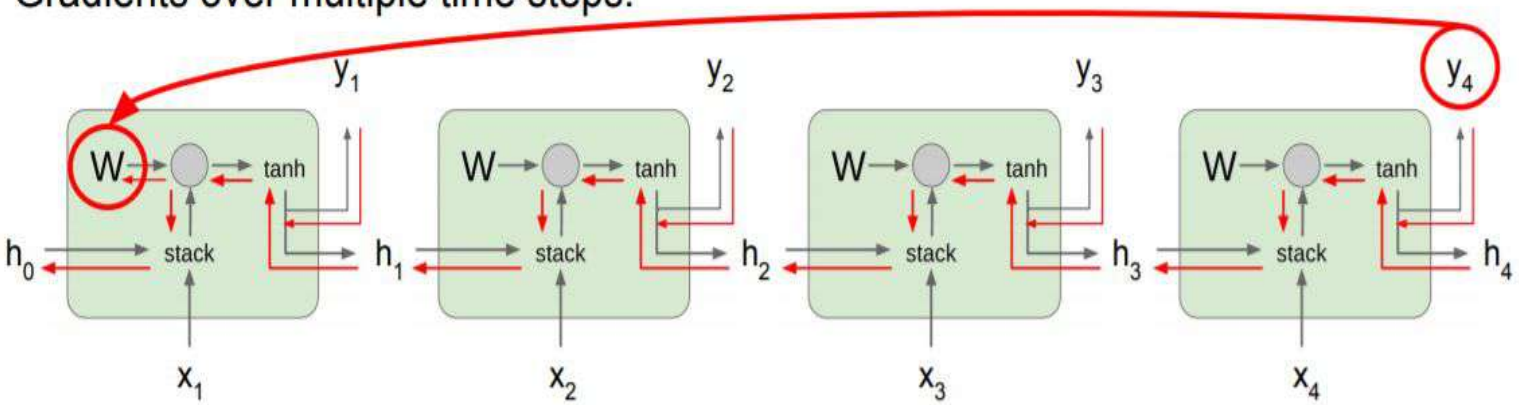


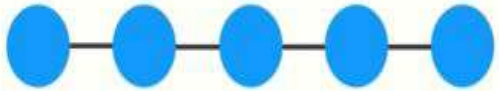
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$
$$= \tanh\left((W_{hh} \quad W_{hx})\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$
$$= \tanh\left(W\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$

$$\frac{\partial h_t}{\partial h_{t-1}} = tanh'(W_{hh}h_{t-1} + W_{xh}x_t)W_{hh}$$

RNN Gradient Flow

Gradients over multiple time steps:





$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^{T} \frac{\partial h_t}{\partial h_{t-1}} \right) \frac{\partial h_1}{\partial W}$$

Bengio et al, “Learning long-term dependencies with gradient descent is difficult”, IEEE Transactions on Neural Networks, 1994

Pascanu et al, “On the difficulty of training recurrent neural networks”, ICML 2013

Vanishing Gradient Problem

- Lets say the value decreased like 0.863 →0.532 →0.356 →0.192 →0.117 →0.086 →0.023 →0.019..
- You can see that there is no much change in last 3 iterations.
- Multiply many tanh gradients together which are potentially much less than zero — this leads to a vanishing gradient problem.

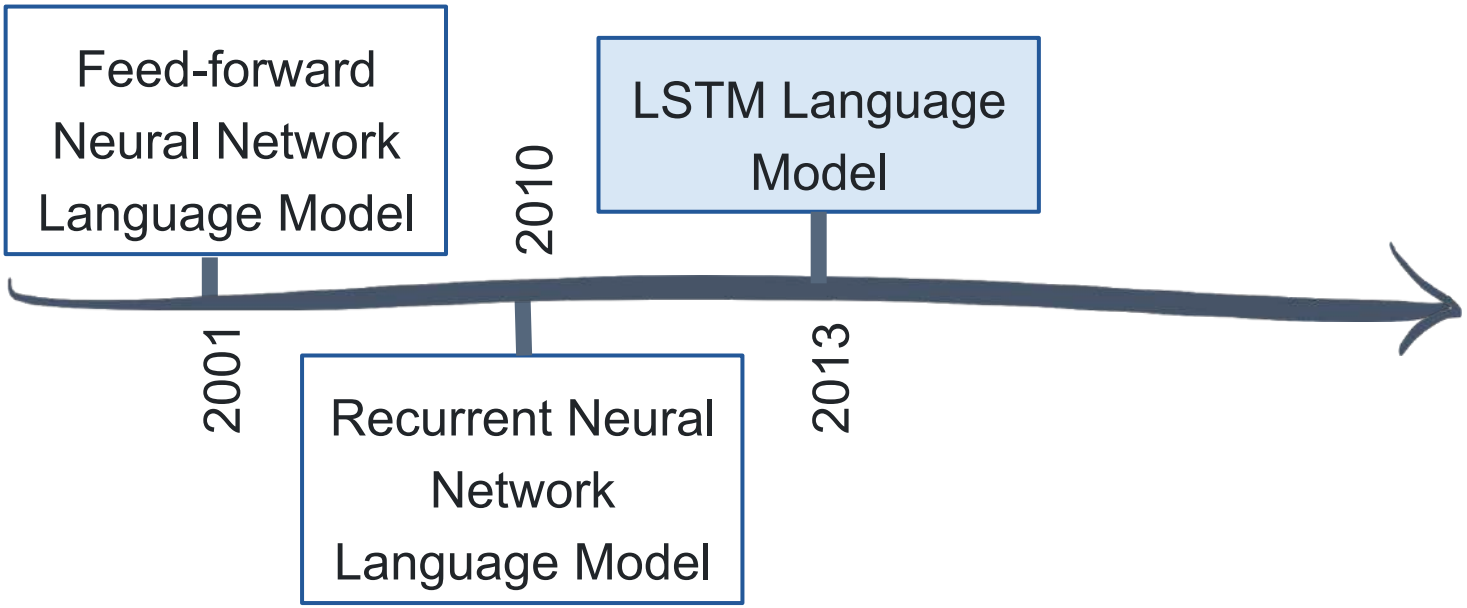


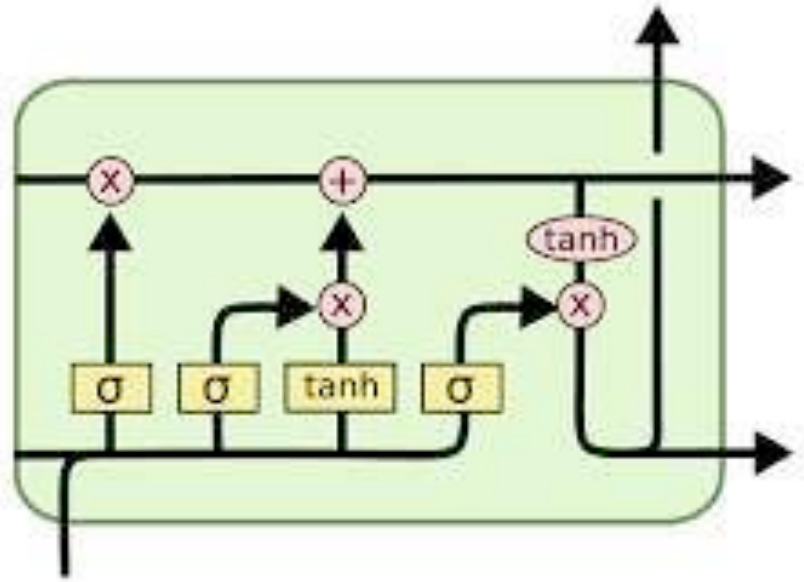
Vanishing Gradients vs Exploding Gradients

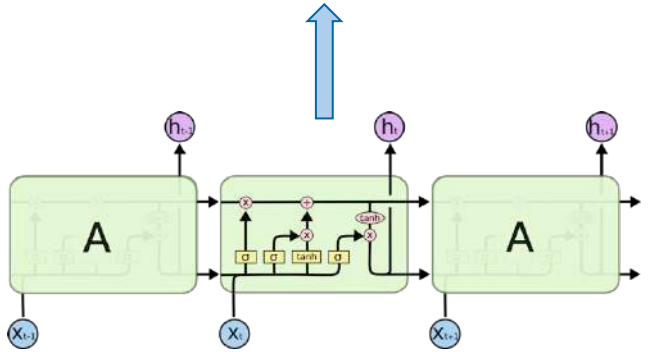
- What if we assumed no non-linearity?
  - Largest singular value > 1: Exploding gradients
    - Solution: Gradient clipping - Scale the gradient if its norm is too big
  - Largest singular value < 1: Vanishing gradients
    - Solution: Change RNN architecture

A History Lesson

- Initially introduced in 1997 [Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural computation]
- Address the vanishing gradient problem
- Model longer context - provide better results



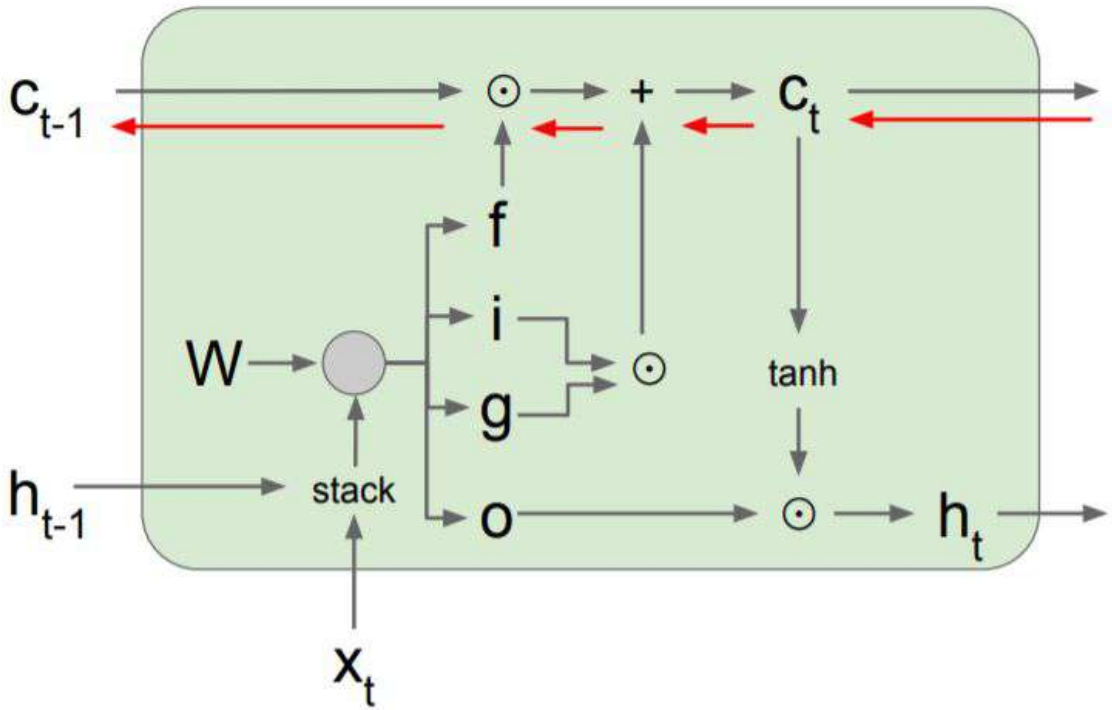




[3] Graves, A., Jaitly, N., & Mohamed, A. R. (2013, December). Hybrid speech recognition with deep bidirectional LSTM. In Automatic Speech Recognition and Understanding (ASRU), 2013 IEEE Workshop on (pp. 273-278). IEEE.

LSTM (Long Short-Term Memory Networks)

i: Input gate, whether to write to cell
f: Forget gate, Whether to erase cell
o: Output gate, How much to reveal cell
g: Gate gate (?), How much to write to cell

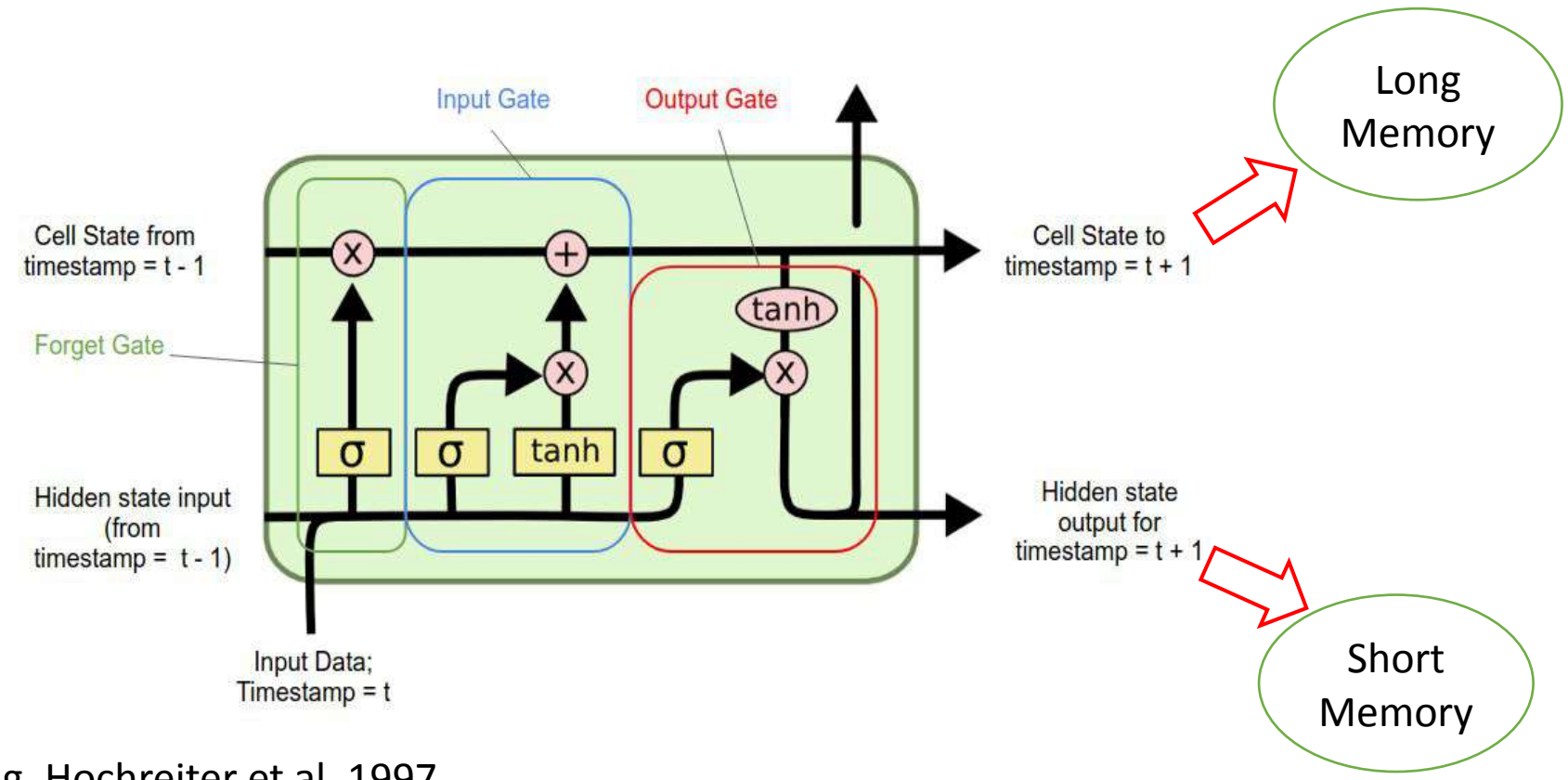


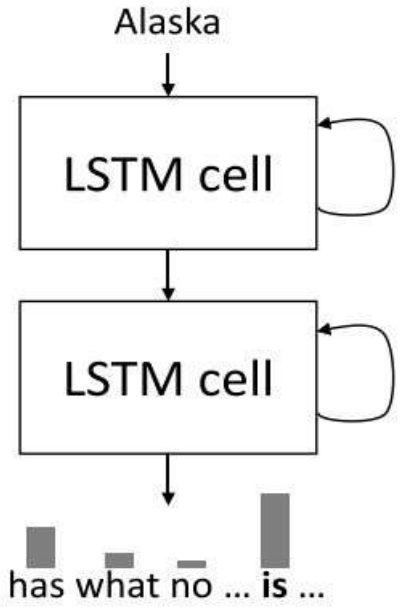
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

LSTM

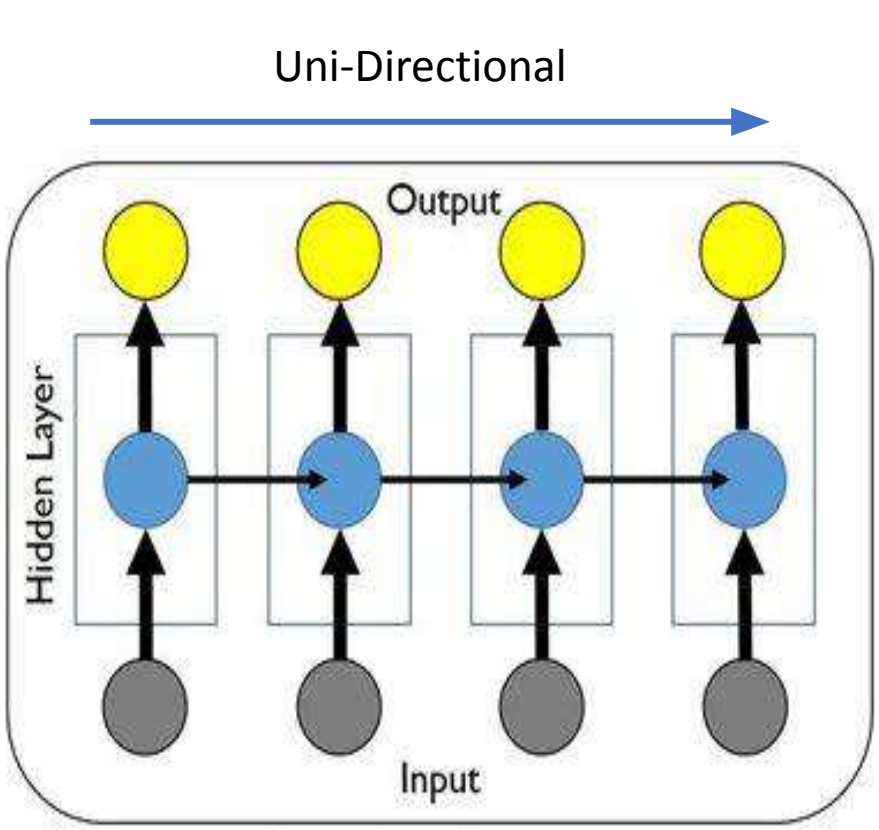




LSTM

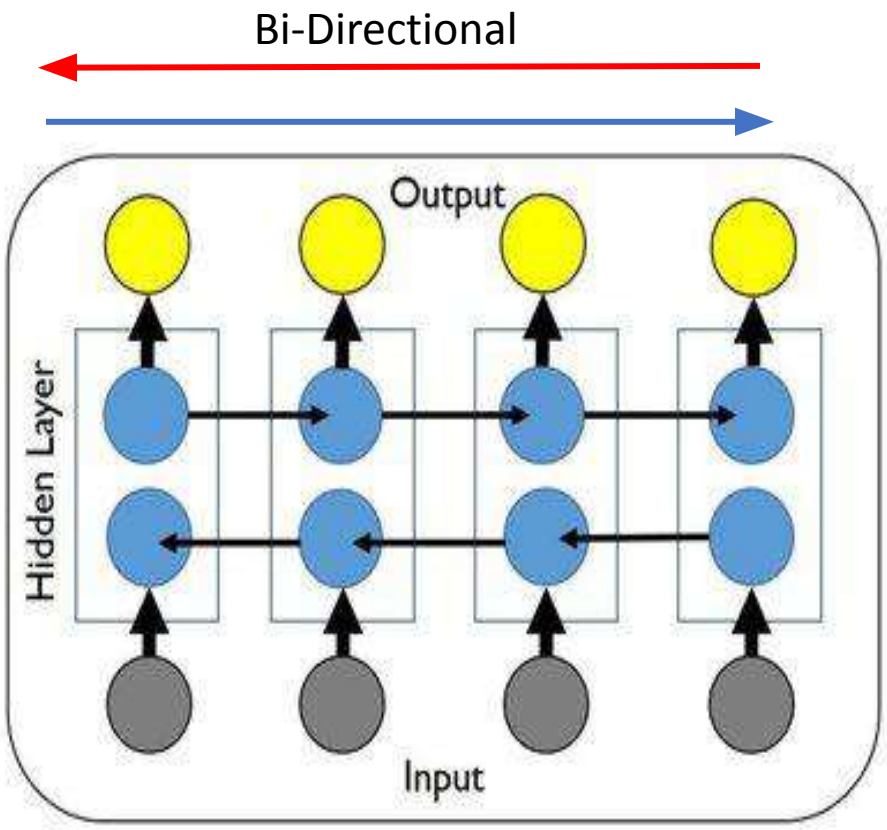
e.g. Hochreiter et al. 1997

LSTM vs BiLSTM



LSTM Architecture

Hochreiter & Schmidhuber, 1997



BiLSTM Architecture

Graves & Schmidhuber, 2005

Do LSTMs solve the vanishing gradient problem?

- The LSTM architecture makes it easier for the RNN to preserve information over many timesteps
  - e.g. if the f = 1 and the i = 0, then the information of that cell is preserved indefinitely.
  - By contrast, it's harder for vanilla RNN to learn a recurrent weight matrix Wh that preserves info in hidden state
- LSTM doesn't guarantee that there is no vanishing/exploding gradient, but it does provide an easier way for the model to learn long-distance dependencies

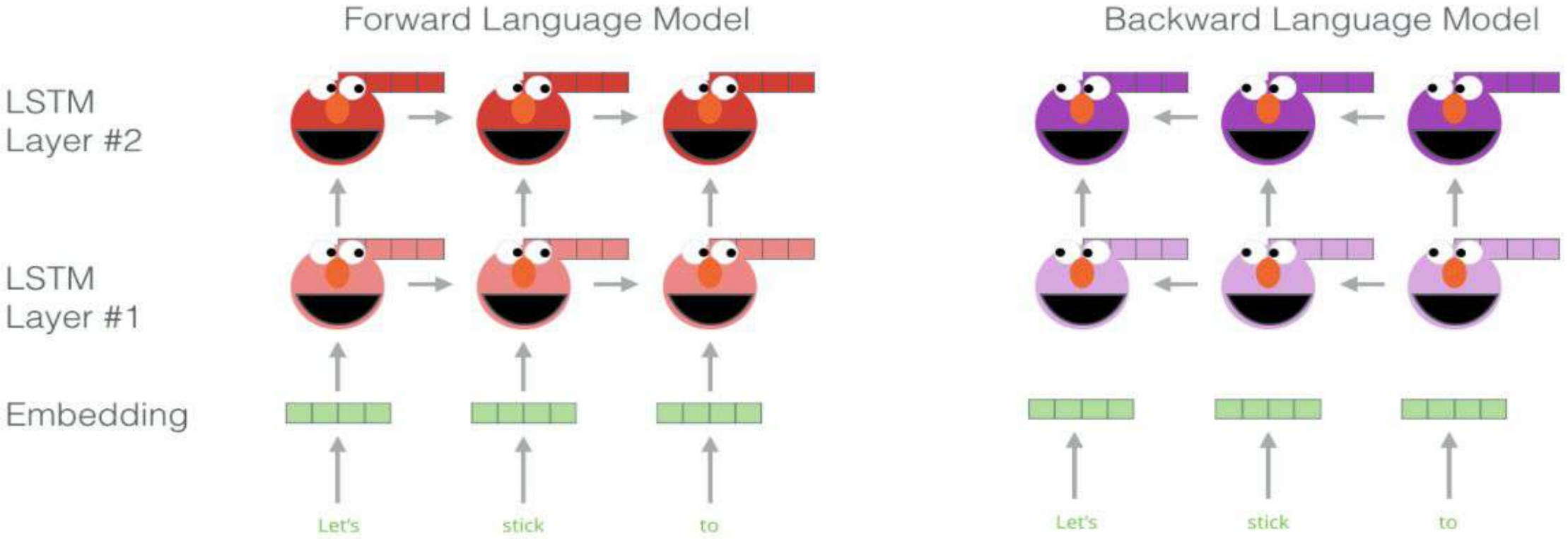


Using Neural Language Models for Word Representation

Inspired by the lectures of Julia Tsvetkov (CMU), Jacob Eisenstein (Georgia Tech) and David Bamman (Berkeley)

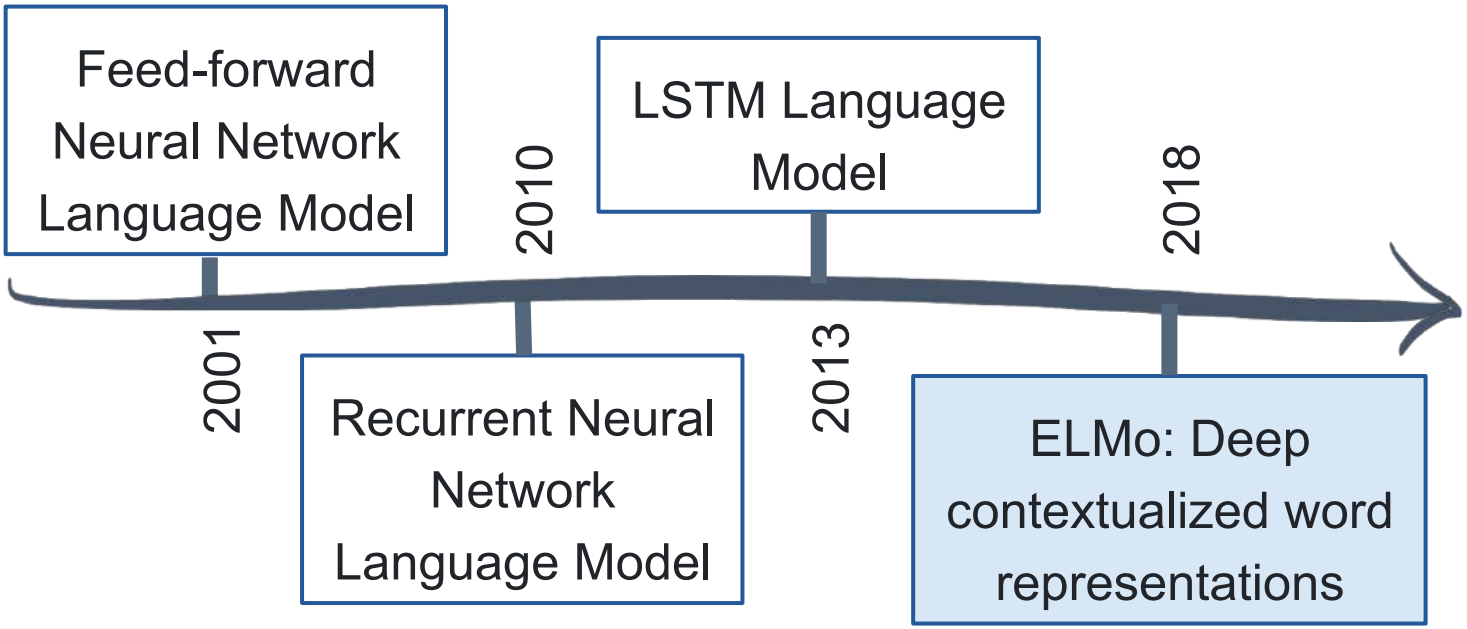
Contextualized word representations (ELMo)

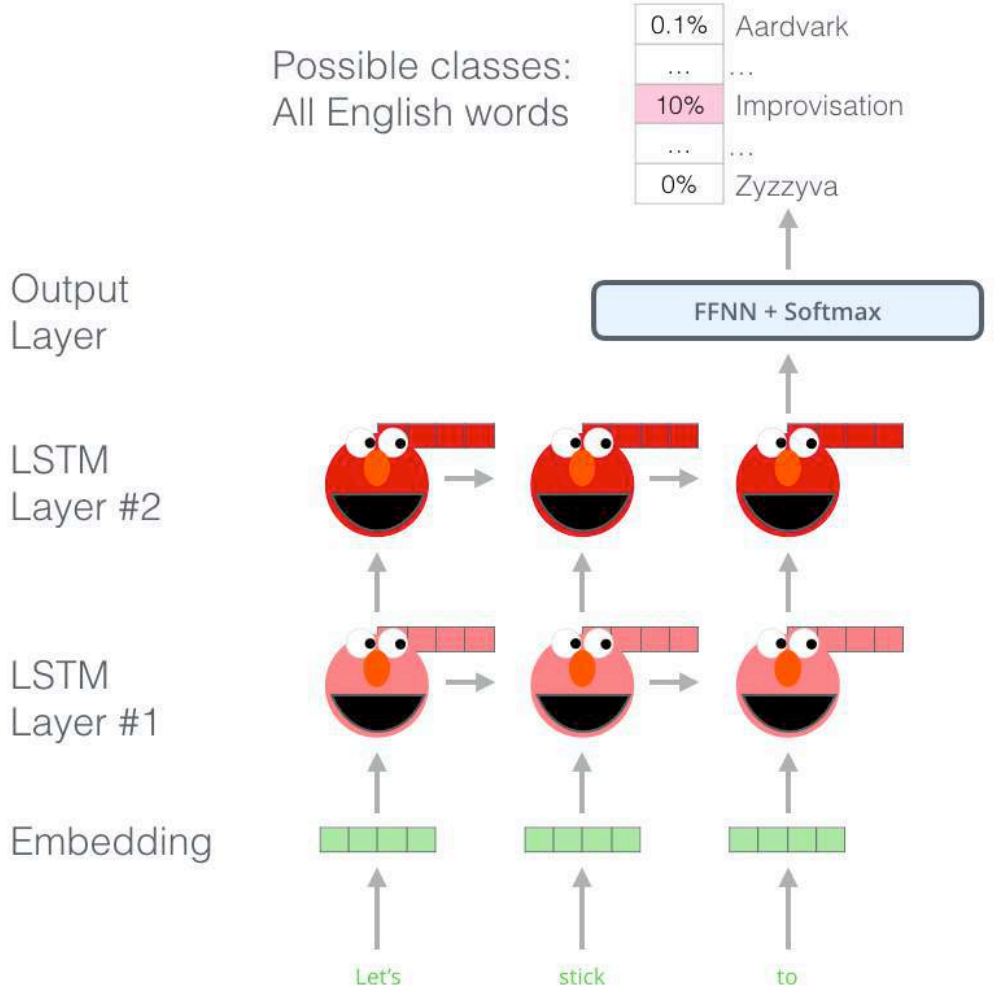
- “Instead of using a fixed embedding for each word, ELMo looks at the entire sentence before assigning each word in it an embedding.”
- ELMo goes a step further and trains a bi-directional LSTM – so that its language model doesn’t only have a sense of the next word, but also the previous word.
- Learn word embeddings through building bidirectional language models (biLMs).



http://jalammar.github.io/illustrated-bert/

A History Lesson

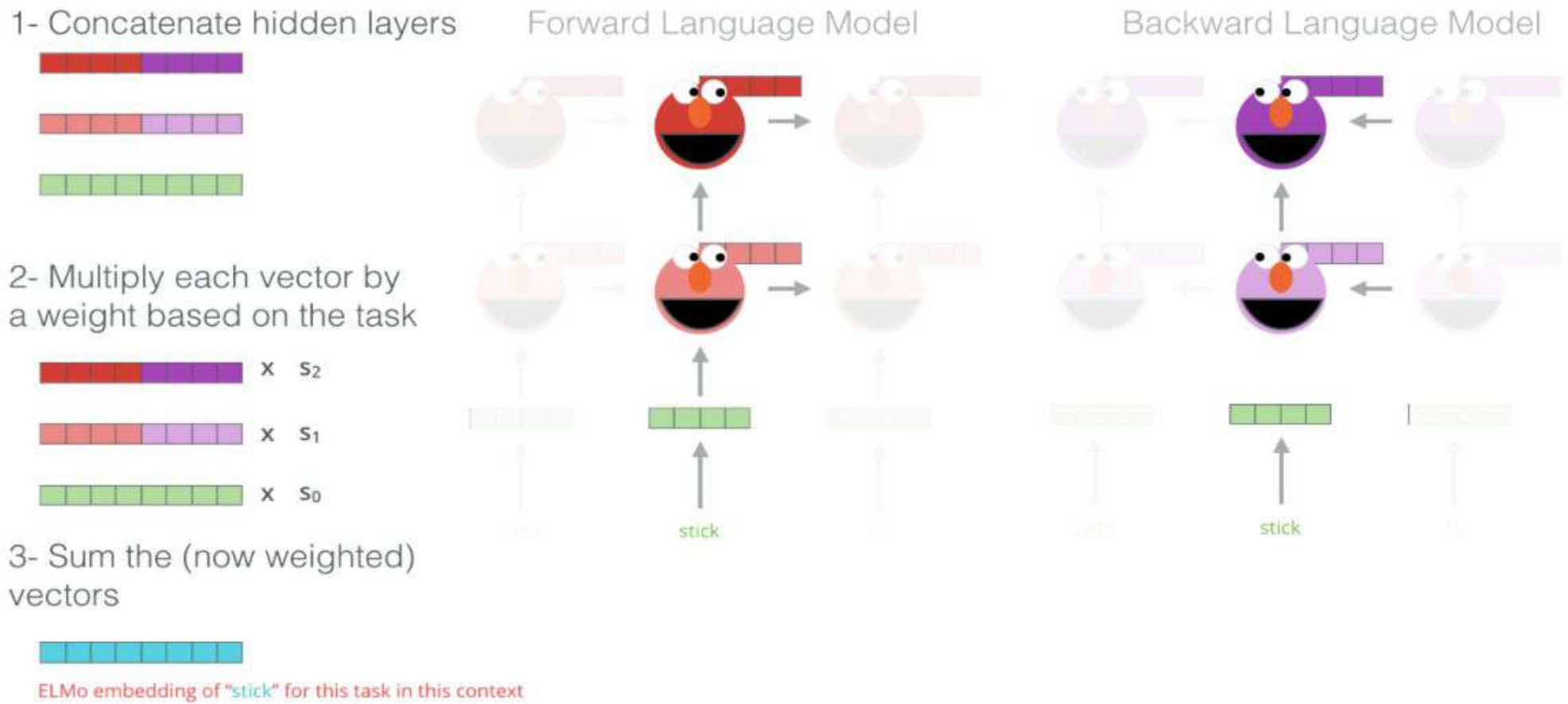




[3] Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, Luke Zettlemoyer, Deep contextualized word representations. NAACL 2018

ELMo

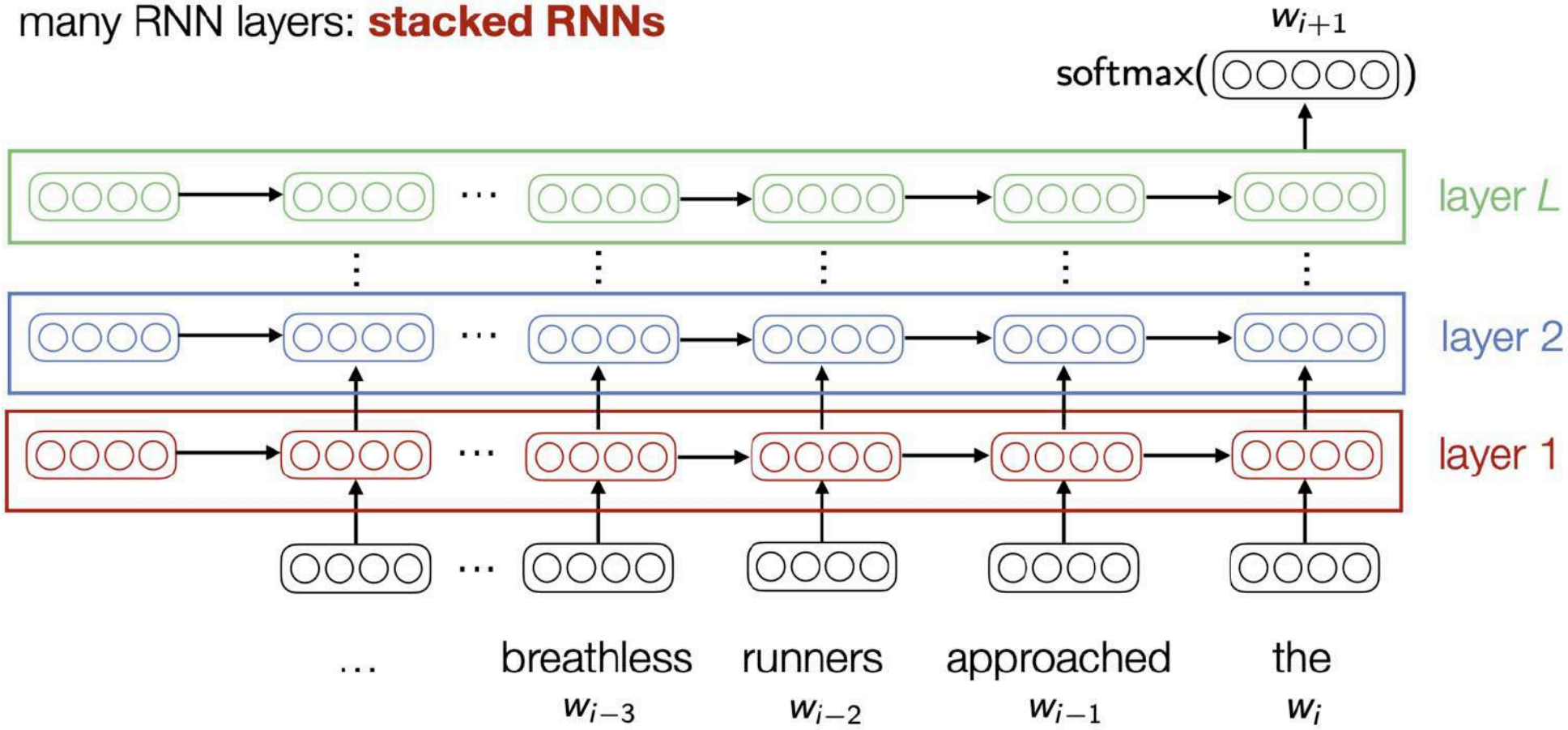
- ELMo produces the contextualized embedding through grouping together the hidden states (and initial embedding) in a certain way (concatenation followed by weighted summation).



http://jalammar.github.io/illustrated-bert/

Stacking recurrent layers

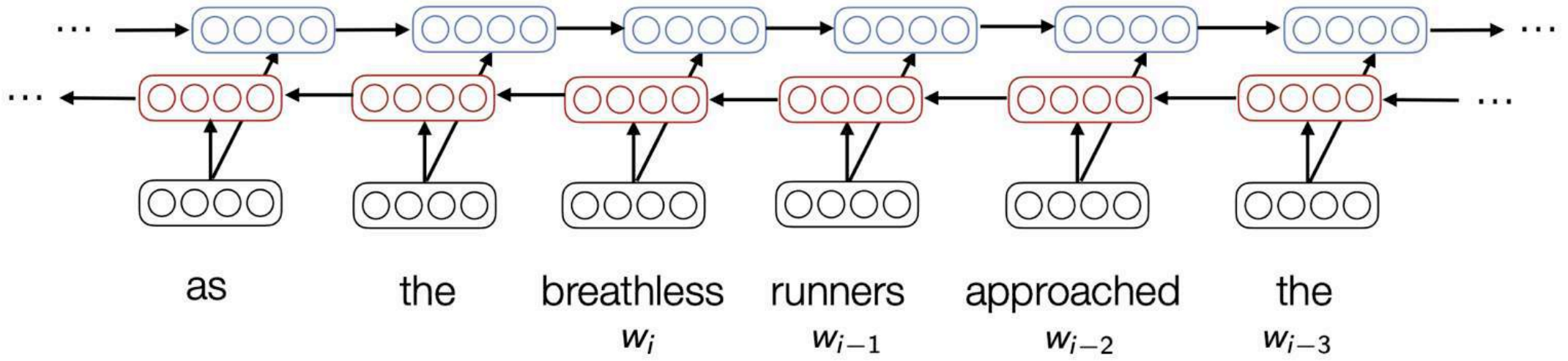
Just as we can add many hidden layers to feed-forward networks, can also add many RNN layers: **stacked RNNs**



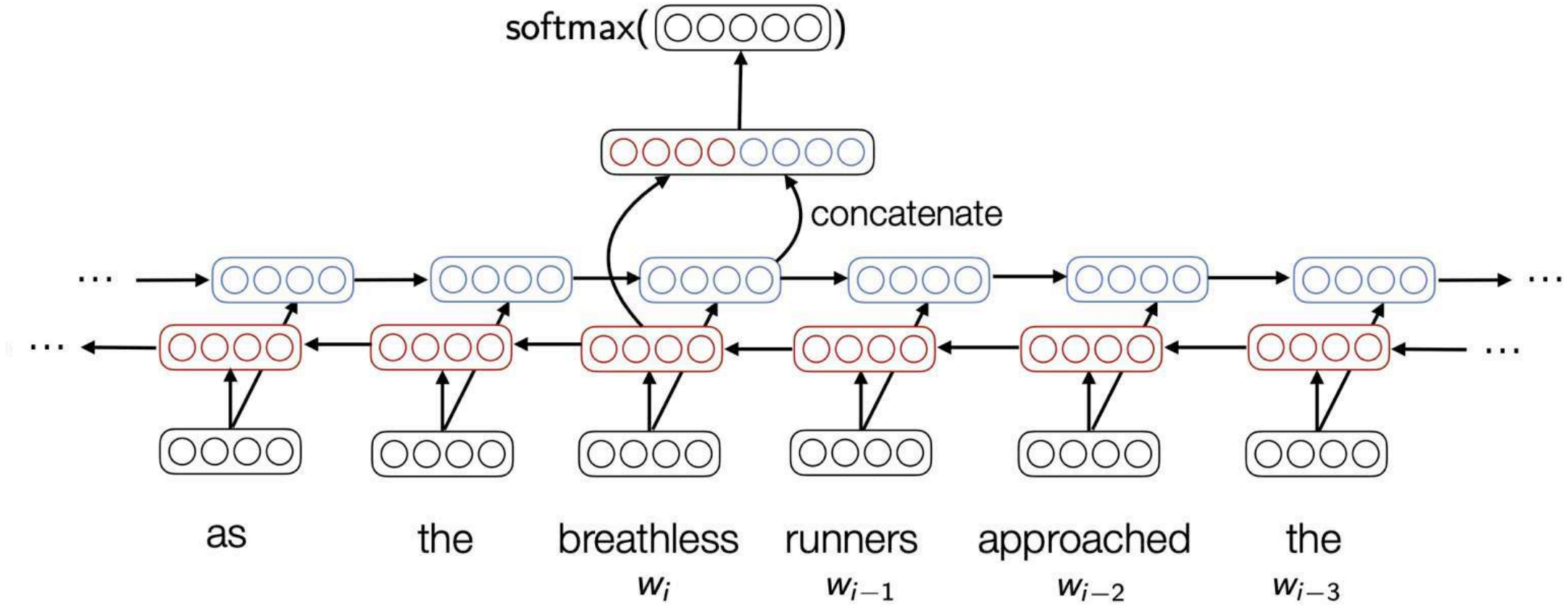
Bidirectional RNN

In language it is also often useful to model past **and** future context

- We can also run an RNN in the backward direction (reverse reading order)
- Combining both directions works best



Bidirectional RNN

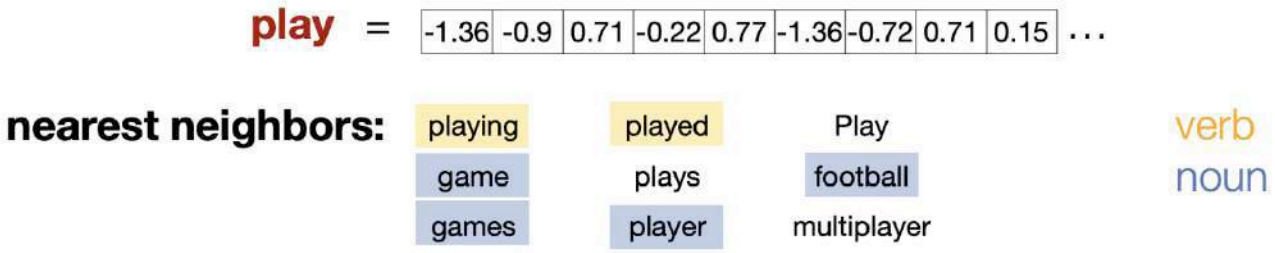


Using LMs as word representations

the new-look **play** area is due to be completed by early spring 2020

gerrymandered congressional districts favor representatives who **play** to the party base

the freshman then completed the three-point **play** for a 66-63 lead



Homonyms:

She **left** her book in her locker last night.
My **left** foot really hurts.

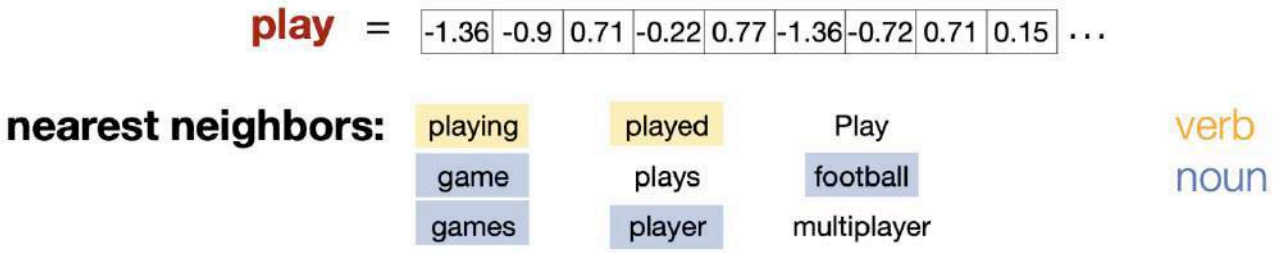
- **Issue** with word2vec:
  one representation for all senses and part-of-speech of the same word

Using LMs as word representations

the new-look **play** area is due to be completed by early spring 2020

gerrymandered congressional districts favor representatives who **play** to the party base

the freshman then completed the three-point **play** for a 66-63 lead

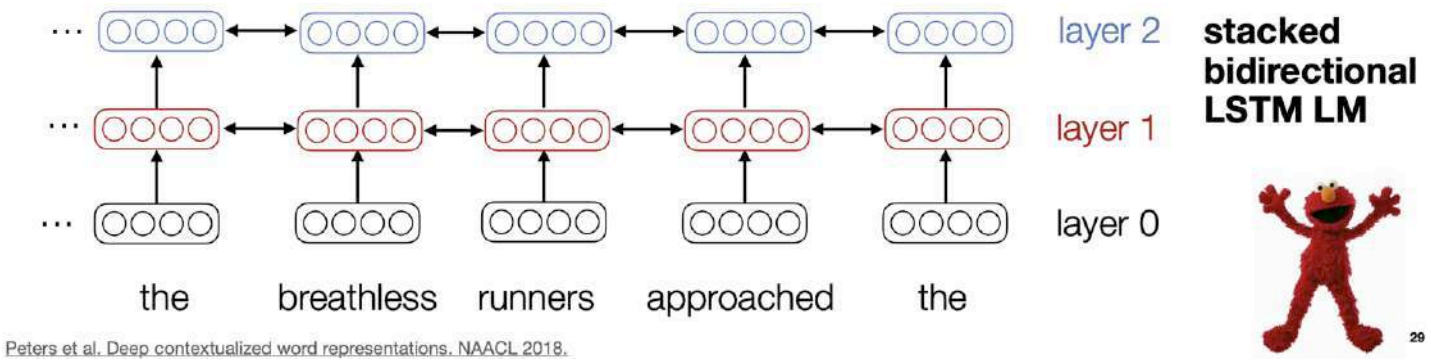


Homonyms:

She **left** her book in her locker last night.
My **left** foot really hurts.

- **Issue** with word2vec:
  one representation for all senses and part-of-speech of the same word
- **Idea**: we want ***contextualized*** word embeddings instead
  = representations of a word that differ depending on the context, but still leverage the unsupervised training on massive data
- **Solution**:
  use the hidden layer representations from a neural language model.

ELMo: Deep contextualized word embeddings

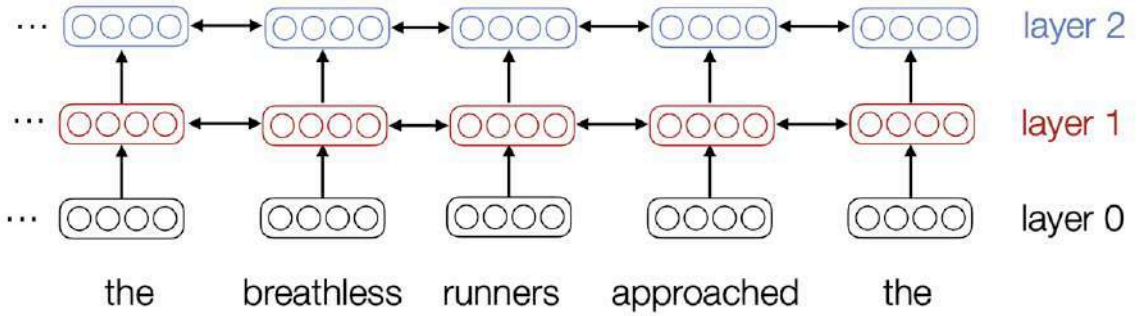


Peters et al. Deep contextualized word representations. NAACL 2018.

- ELMo = “Embeddings from Language Models”
- Paper: https://arxiv.org/pdf/1802.05365.pdf
- **Idea**: we want ***contextualized*** word embeddings instead
  = representations of a word that differ depending on the context, but still leverage the unsupervised training on massive data
- **Solution**:
  use the hidden layer representations from a neural language model.

ELMo: Deep contextualized word embeddings





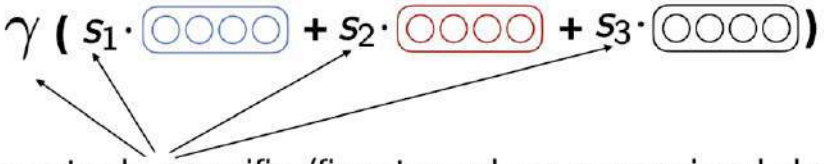
Peters et al. Deep contextualized word representations. NAACL 2018.



- ELMo = “Embeddings from Language Models”
- Paper: https://arxiv.org/pdf/1802.05365.pdf
- **Idea**: we want ***contextualized*** word embeddings instead
  = representations of a word that differ depending on the context, but still leverage the unsupervised training on massive data
- **Solution**:
  use the hidden layer representations from a neural language model.

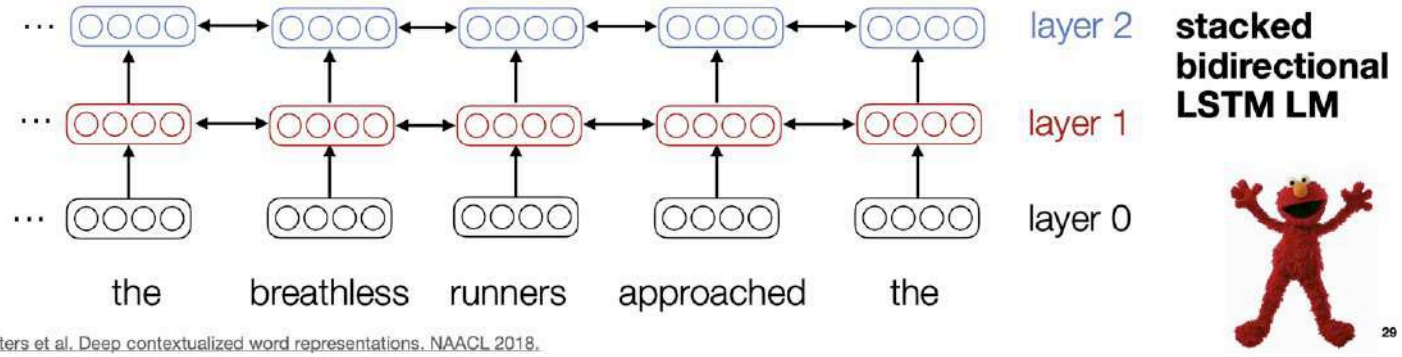
ELMo: Deep contextualized word embeddings

Key idea: context-dependent embedding for each word interpolates representations for that word from each layer



$$\sum_{j=1}^{L} s_j = 1$$

Interpolation weights are task-specific (fine-tuned on supervised data.)



stacked bidirectional LSTM LM

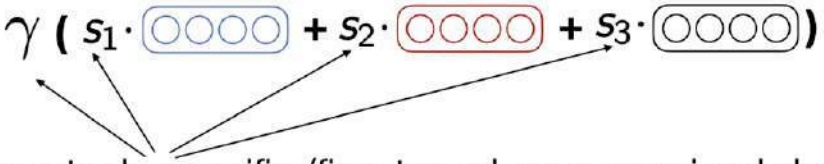
Peters et al. Deep contextualized word representations. NAACL 2018.

29

- ELMo = "Embeddings from Language Models"
- Paper: https://arxiv.org/pdf/1802.05365.pdf
- **Idea**: we want ***contextualized*** word embeddings instead
  = representations of a word that differ depending on the context, but still leverage the unsupervised training on massive data
- **Solution**:
  use the hidden layer representations from a neural language model.

ELMo: Deep contextualized word embeddings

Key idea: context-dependent embedding for each word interpolates representations for that word from each layer



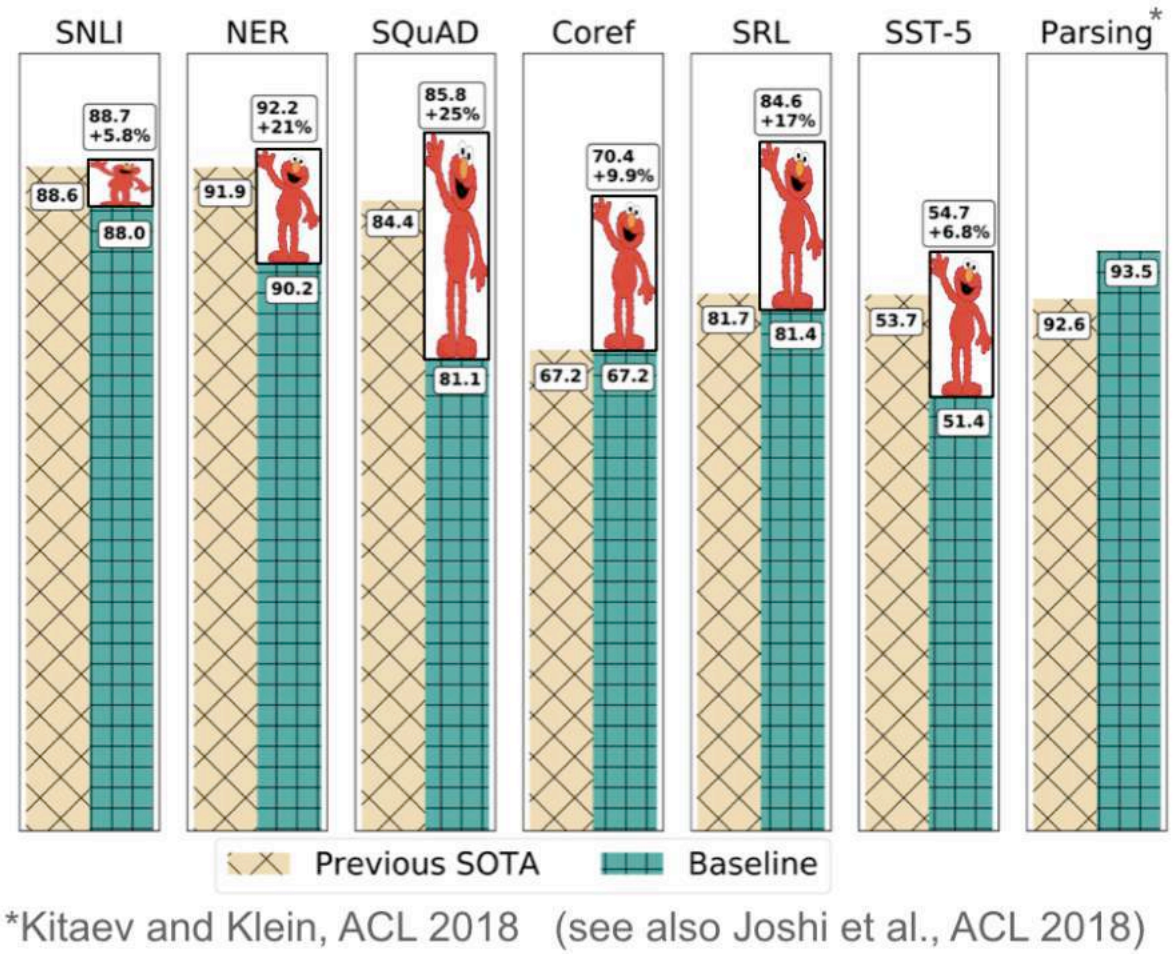
$$\sum_{j=1}^{L} s_j = 1$$

Interpolation weights are task-specific (fine-tuned on supervised data.)

	TASK	BASELINE	ELMO + BASELINE
question answering	SQuAD	81.1	85.8
natural language inference	SNLI	88.0	88.7 ± 0.17
semantic role labeling	SRL	81.4	84.6
coreference	Coref	67.2	70.4
named entity recognition	NER	90.15	92.22 ± 0.10
sentiment analysis	SST-5	51.4	54.7 ± 0.5

- ELMo = “Embeddings from Language Models”
- Paper: https://arxiv.org/pdf/1802.05365.pdf
- **Idea**: we want ***contextualized*** word embeddings instead
  = representations of a word that differ depending on the context, but still leverage the unsupervised training on massive data
- **Solution**:
  use the hidden layer representations from a neural language model.

ELMo: Deep contextualized word embeddings



*Kitaev and Klein, ACL 2018 (see also Joshi et al., ACL 2018)

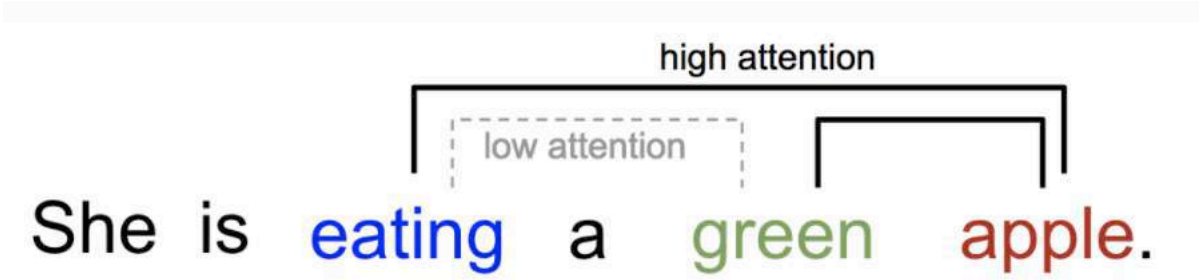
- ELMo = "Embeddings from Language Models"
- Paper: https://arxiv.org/pdf/1802.05365.pdf
- **Idea**: we want ***contextualized*** word embeddings instead
  = representations of a word that differ depending on the context, but still leverage the unsupervised training on massive data
- **Solution**:
  use the hidden layer representations from a neural language model.



Using Attention-Based Neural Language Models for Word Representation

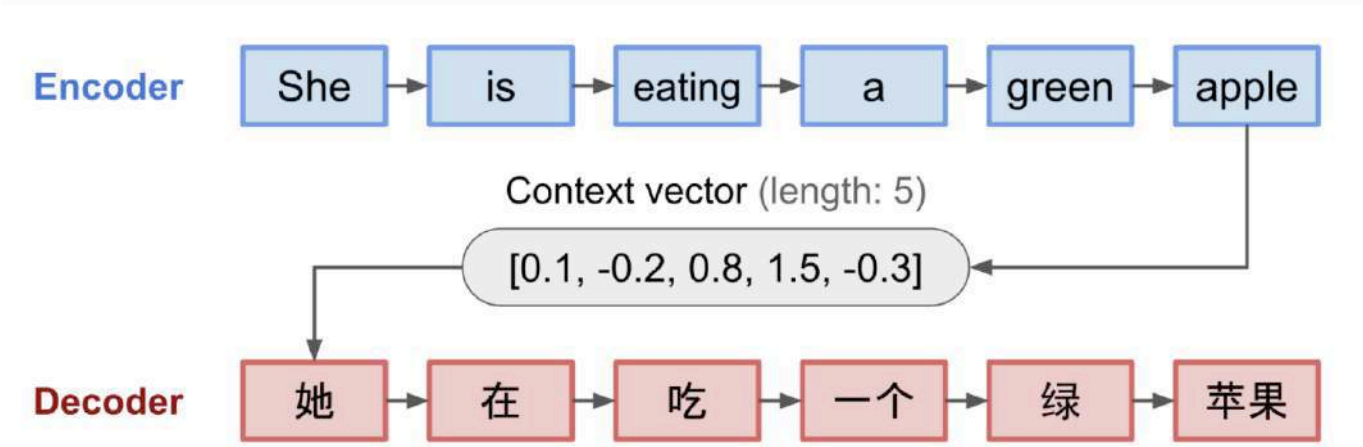
Inspired by the lectures of Julia Tsvetkov (CMU), Jacob Eisenstein (Georgia Tech) and David Bamman (Berkeley)

Attention for text



Idea: Some words in the sentence are more important when trying to interpret new words

Recap: RNN Encoder-Decoder



https://lilianweng.github.io/lil-log/2018/06/24/attention-attention.html

Sequence to sequence modelling (e.g. machine translation) with an encoder-decoder RNN architecture:

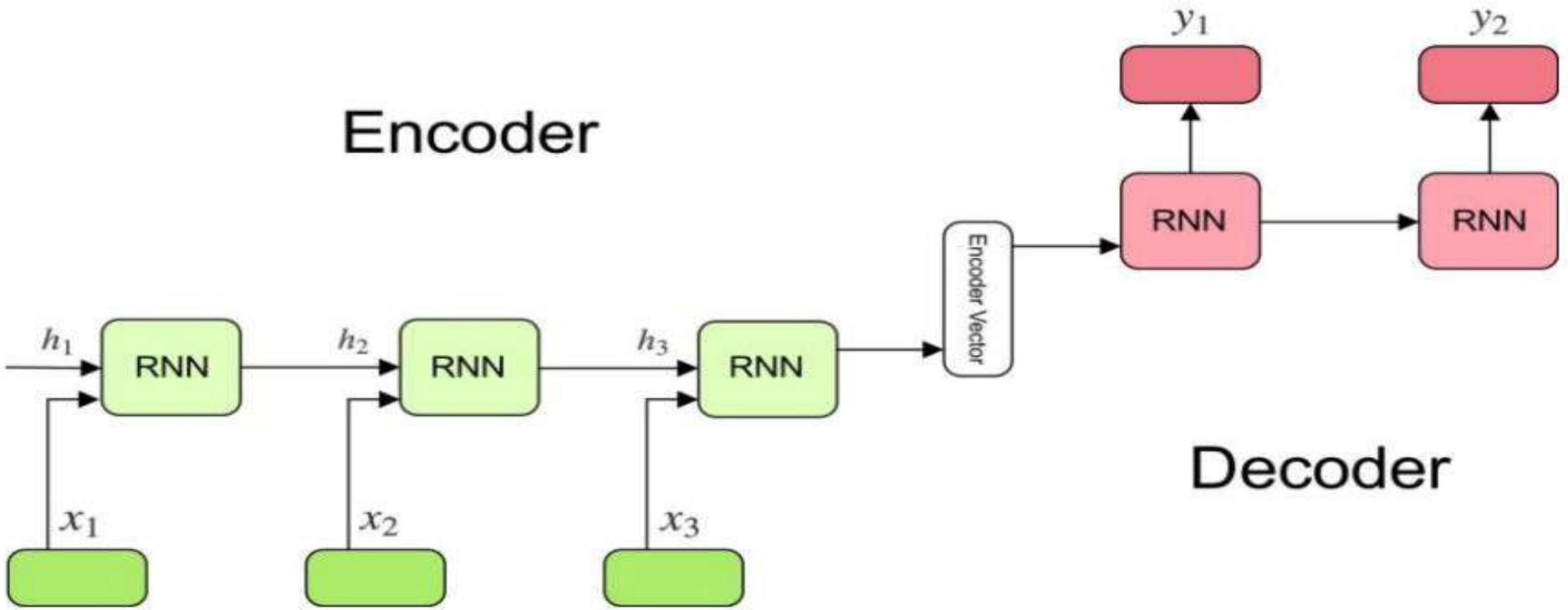
An **encoder** processes the input sequence and compresses the information into a vector (sentence embedding) of a *fixed length*. This representation is expected to be a good summary of the meaning of the *whole* source sequence.

A **decoder** is initialized with the context vector to emit the transformed output. The early work only used the last state of the encoder network as the decoder initial state.

Problem: Capturing long contexts (forgetting)

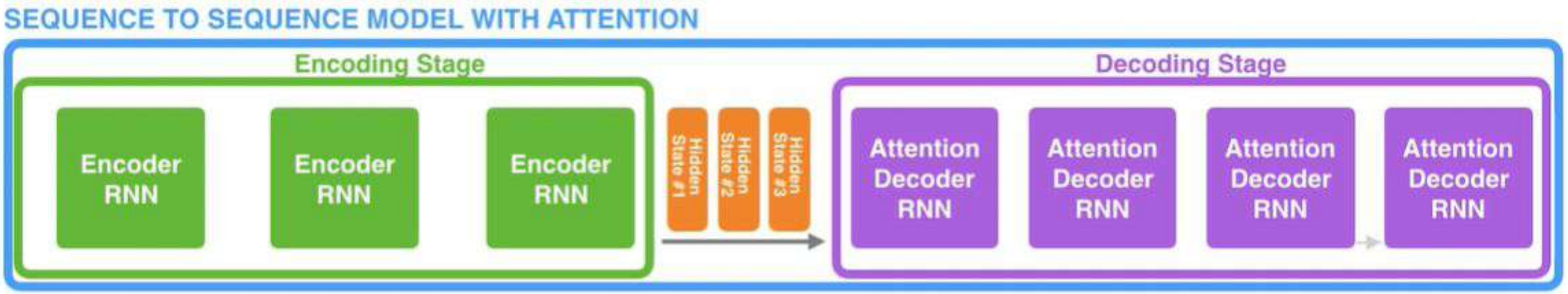
Sequence to Sequence Learning with RNNs

- The model is comprised of two sub-models:
  - **Encoder**: The encoder is responsible for stepping through the input time steps and encoding the entire sequence into a fixed length vector called a context vector.
  - **Decoder**: The decoder is responsible for stepping through the output time steps while reading from the context vector.
- It has a serious problem when working with long sequences.

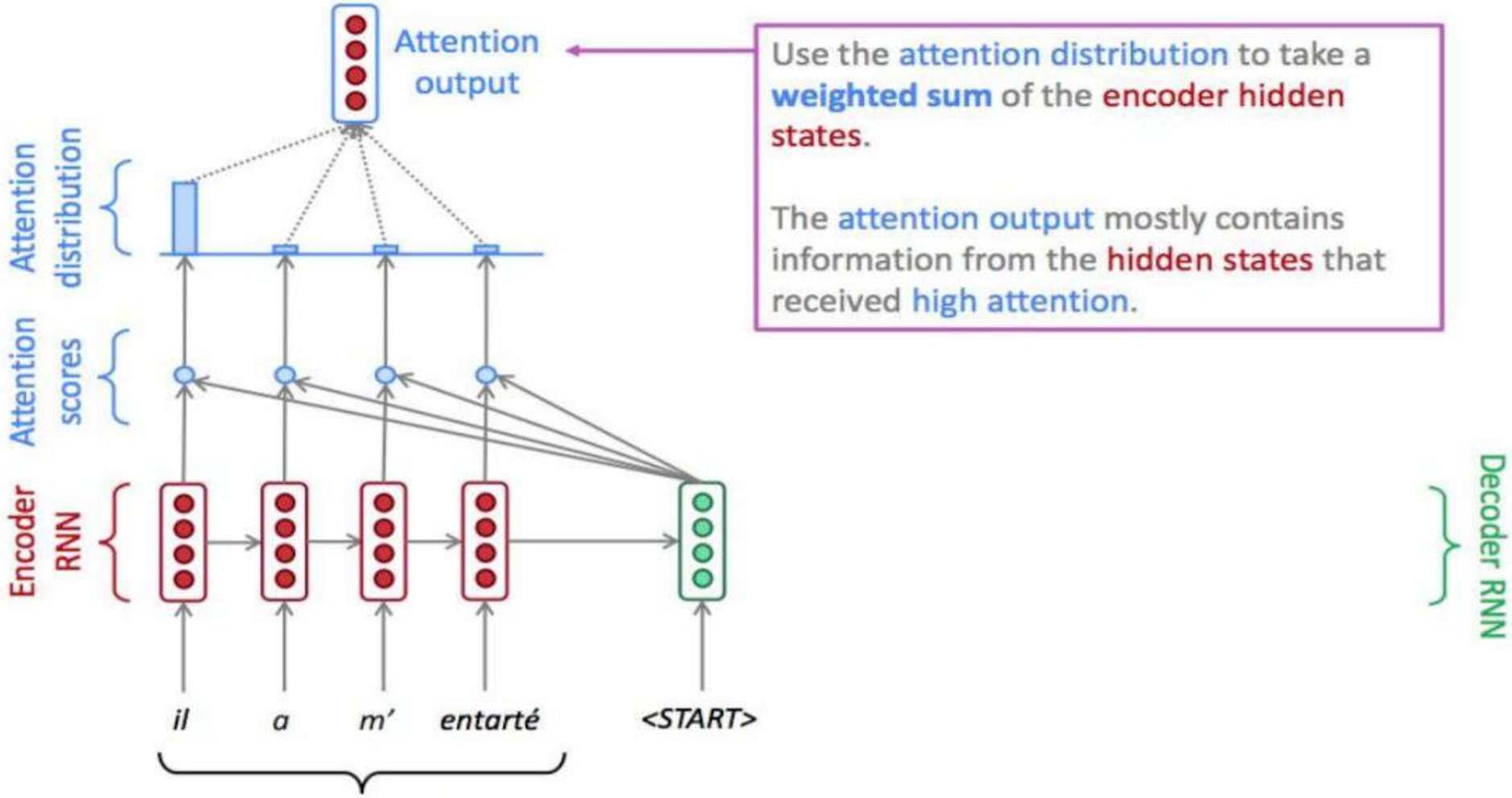


Seq2Seq Model with Attention

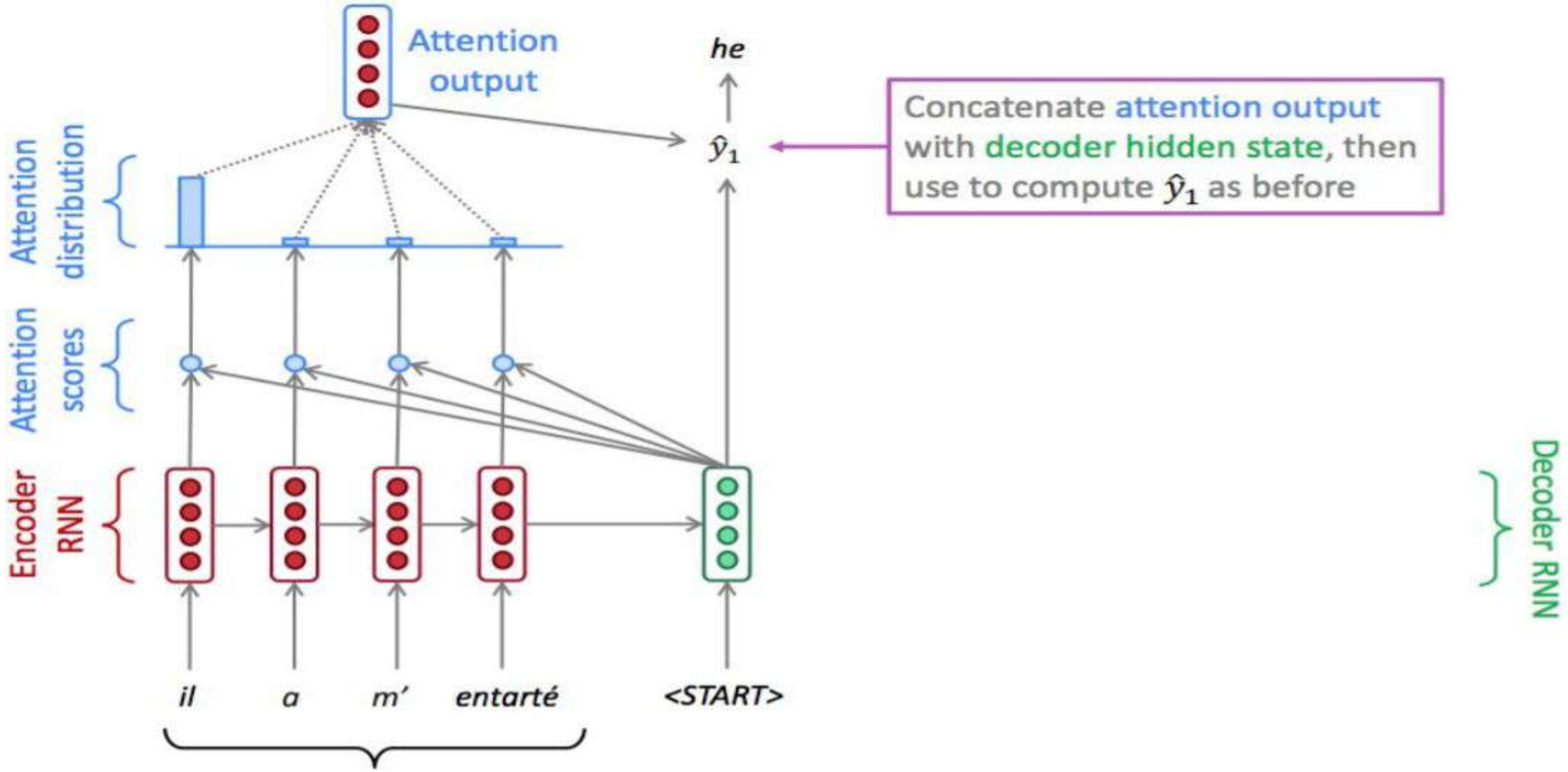
- An attention model differs from a classic sequence-to-sequence model in two main ways
  - Instead of passing the last hidden state of the encoding stage, the encoder passes all the hidden states to the decoder.



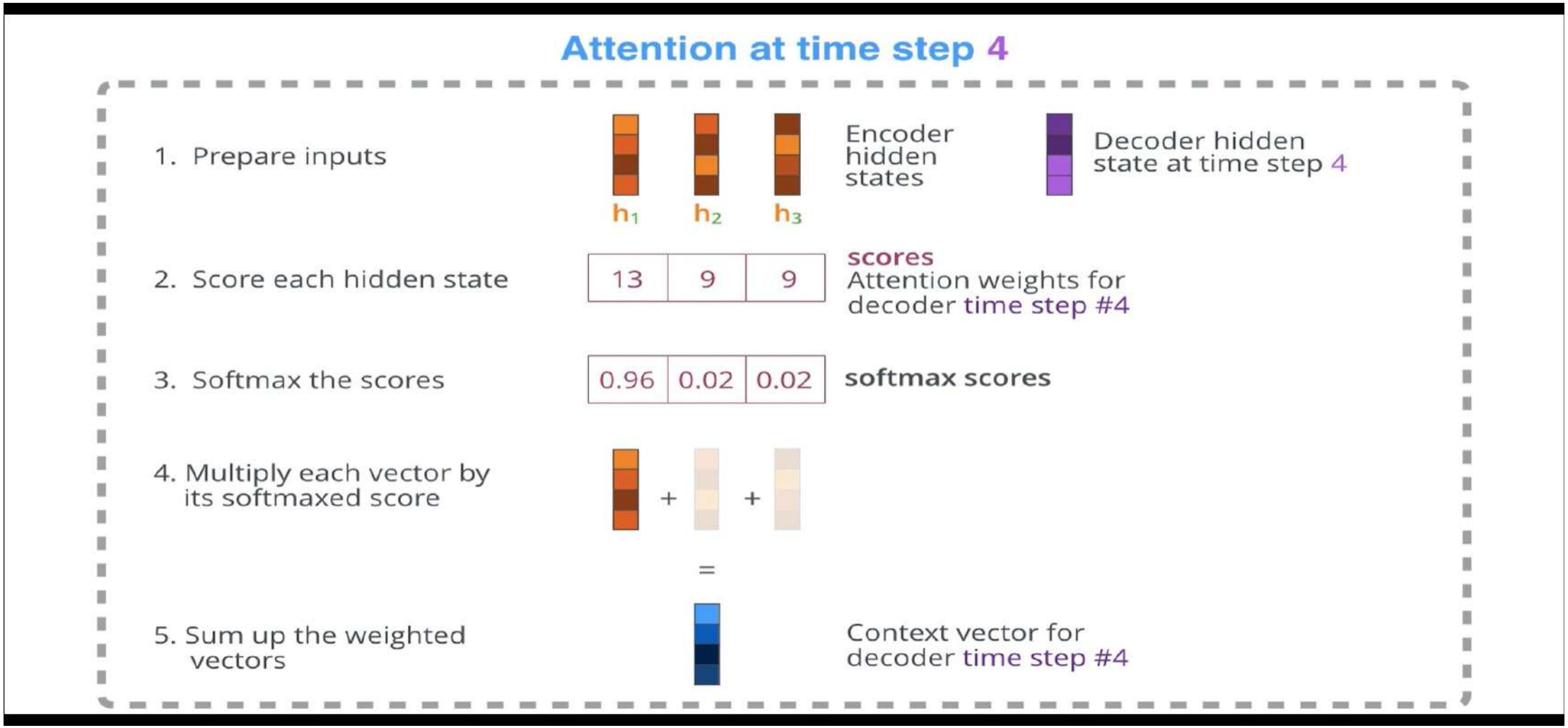
Seq2Seq with Attention



Seq2Seq with Attention

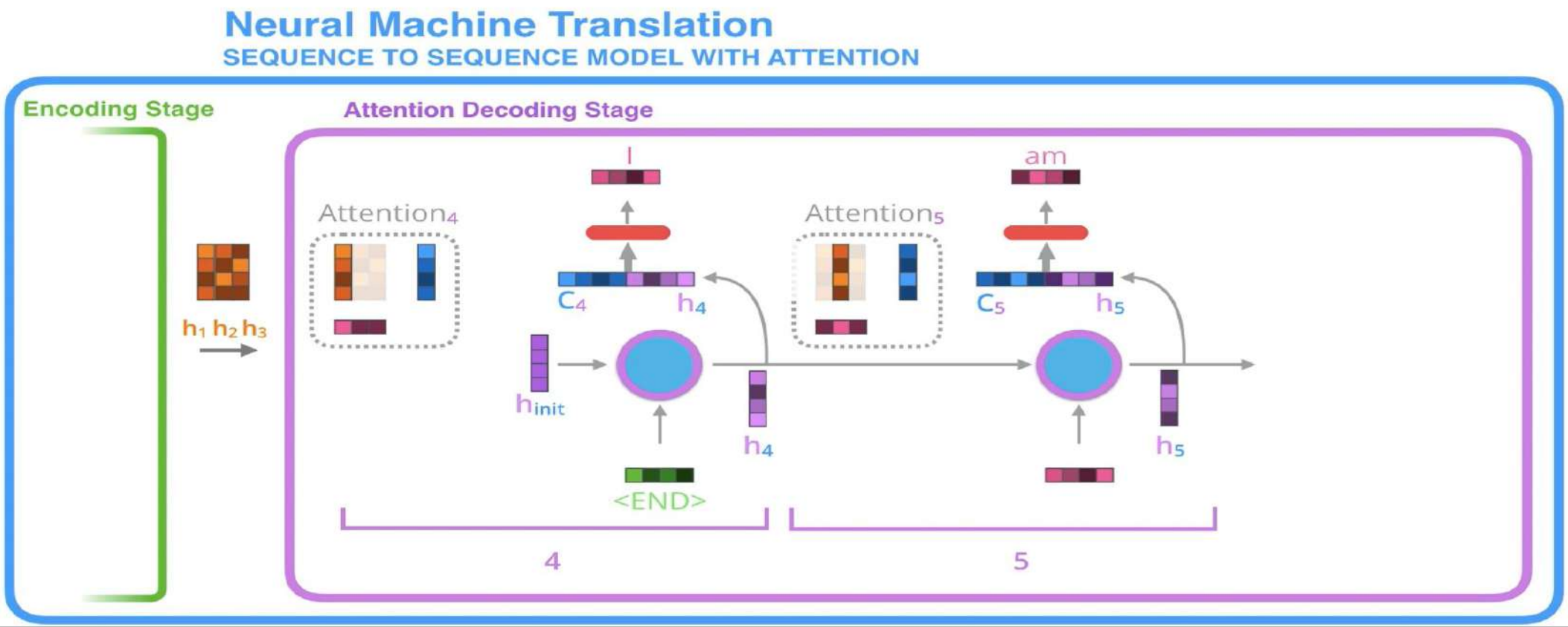


Seq2Seq with Attention



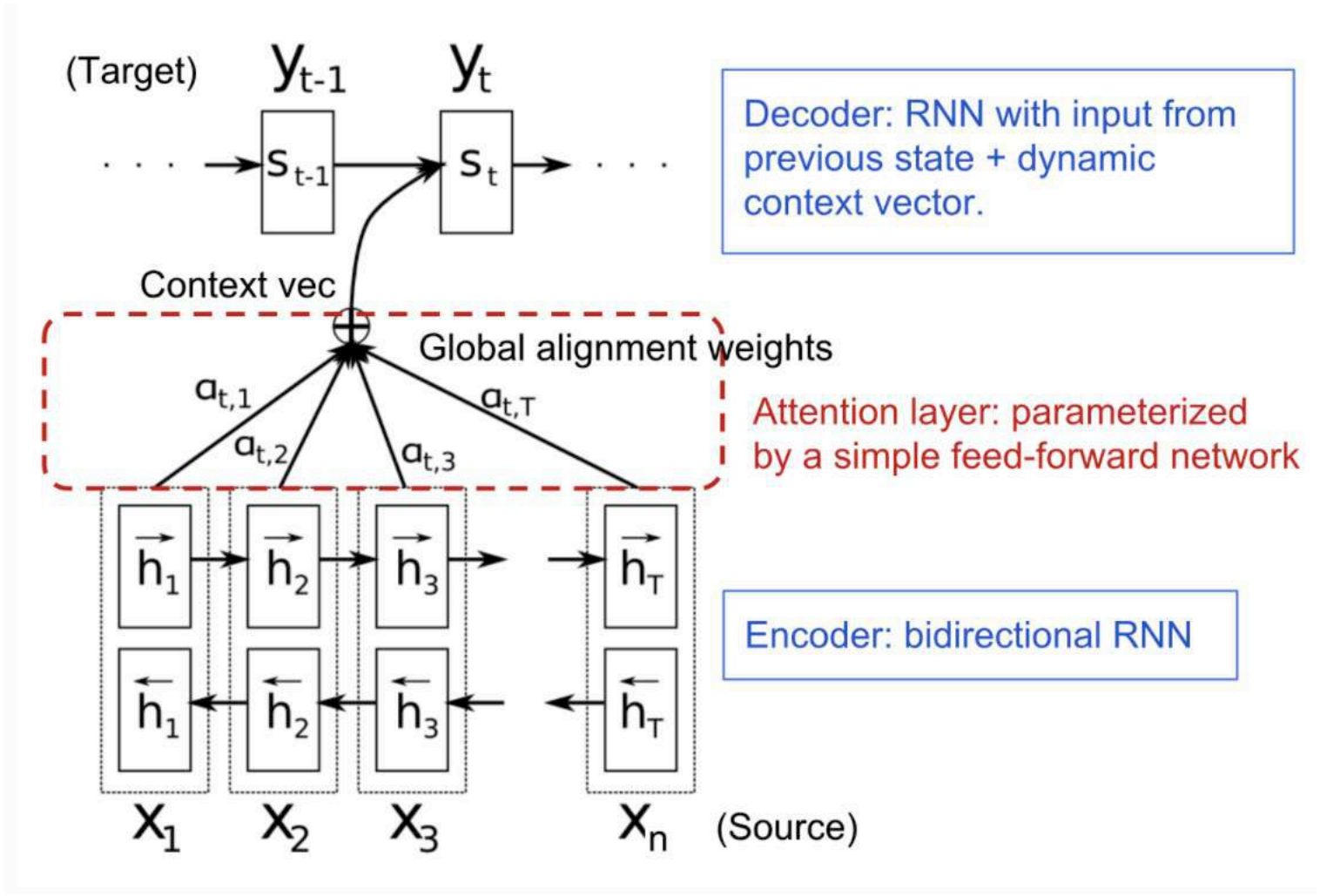
http://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/

Seq2Seq with Attention



http://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/

Attention



https://lilianweng.github.io/lil-log/2018/06/24/attention-attention.html

Idea:

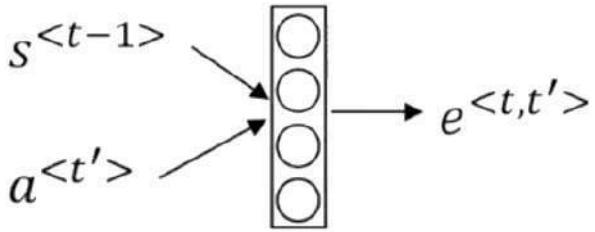
Instead of building a single context vector out of the encoder's last hidden state, create shortcuts between the context vector and the entire source input:

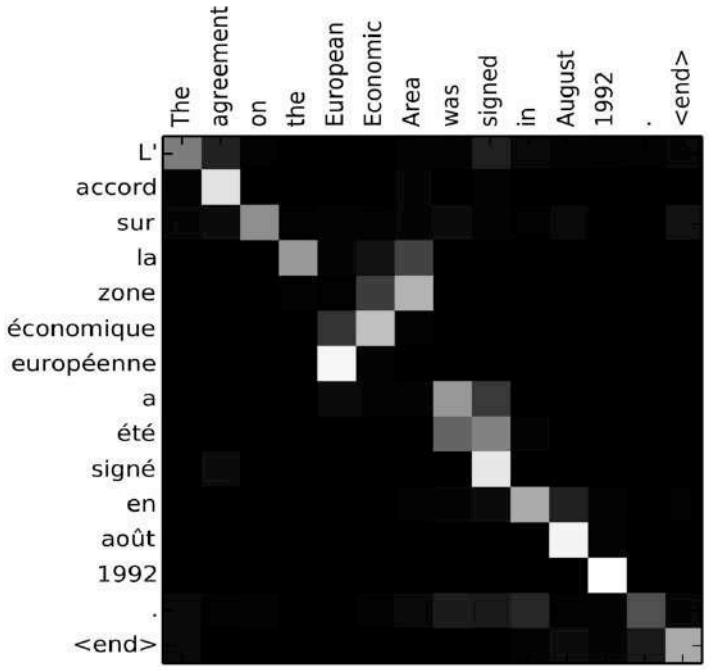
- encoder hidden states
- decoder hidden states
- alignment between source and target

The weights of these shortcut connections are customizable for each output

Computing Attention

$$\alpha^{<t,t'>} = \frac{\exp(e^{<t,t'>})}{\sum_{t'=1}^{T_x} \exp(e^{<t,t'>})}$$





https://lilianweng.github.io/lil-log/2018/06/24/attention-attention.html

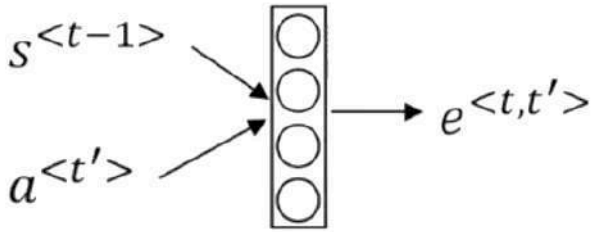
Learn the parameters alpha (the attention weights)

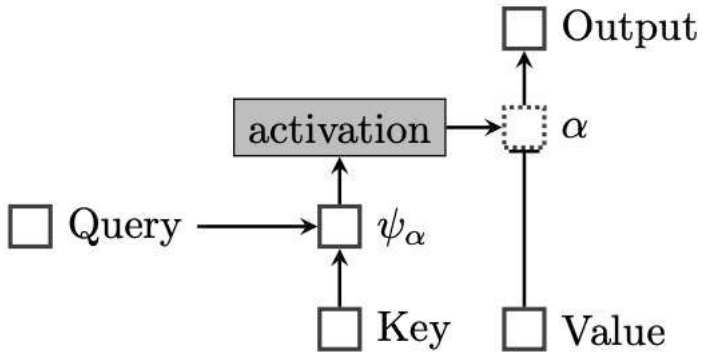
For example just by adding a fully connected DNN with one hidden layer

The goal is to predict how well an input word aligns with an output word (an alignment score $e^{t,t'}$)

Computing Attention

$$\alpha^{<t,t'>} = \frac{\exp(e^{<t,t'>})}{\sum_{t'=1}^{T_x} \exp(e^{<t,t'>})}$$

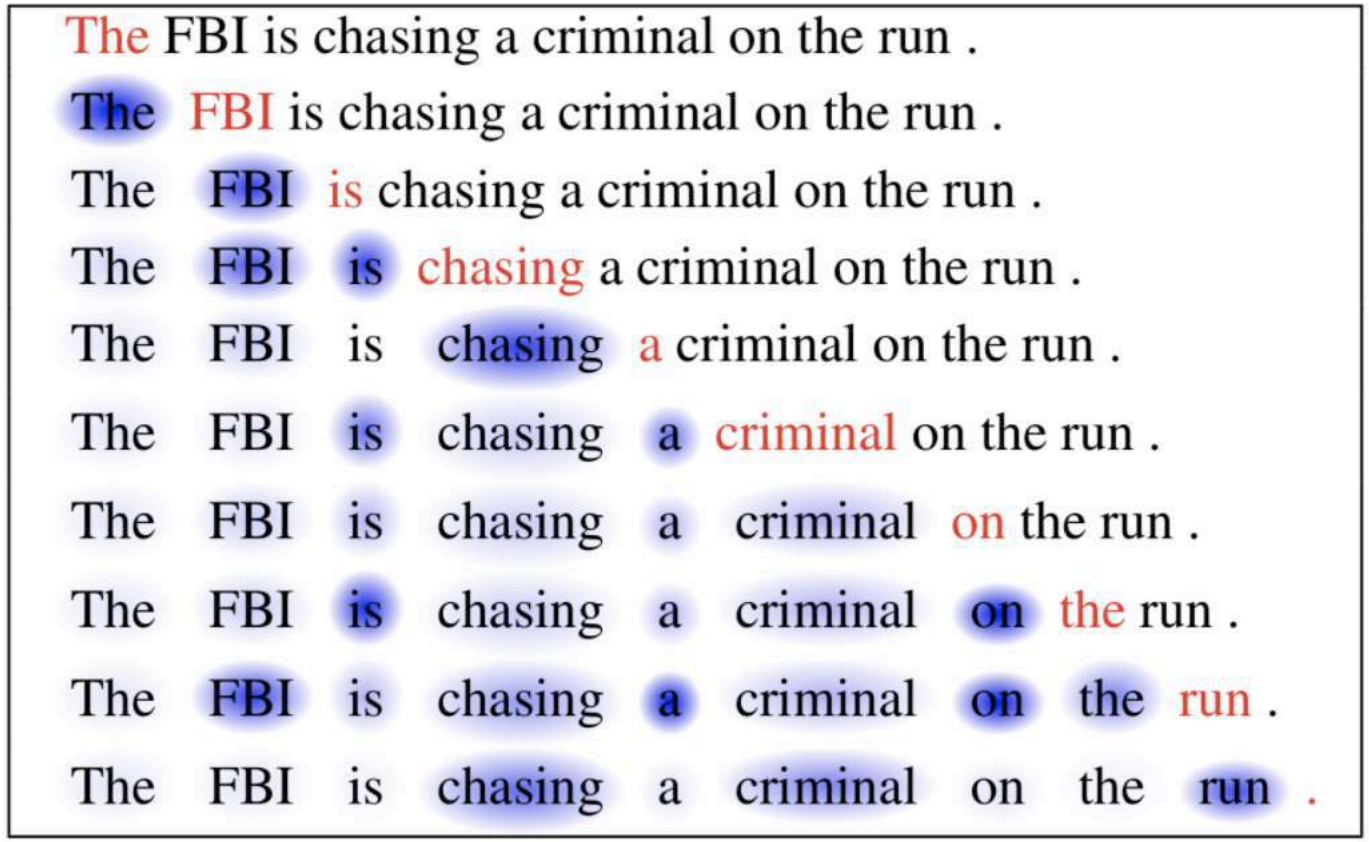




https://lilianweng.github.io/lil-log/2018/06/24/attention-attention.html

- Neural attention mechanism: a weighted average over (word) representations, where the weights are learned, computed as a function of the input.
- Additive attention: ψ is a feed-forward layer with concatenated [k;q] as input.
- Multiplicative attention: ψ is a dot product of linear projections of k, q.

Self-Attention



https://lilianweng.github.io/lil-log/2018/06/24/attention-attention.html

Self-attention is an attention mechanism relating different positions of a single sequence in order to compute a representation of the same sequence

Self-attention is the method the Transformer uses to integrate the “understanding” of other relevant words into the one we’re currently processing.



UNIVERSITÄT BONN

Summary

Neural language models overcome some of the computing issues of probabilistic LMs, leading to a better performance

Recurrent networks state-of-the-art until 2018

Recently, attention networks with transformer architectures perform the best across NLP tasks

These models are very hard to train from scratch, but hidden layers of neural LMs can be used in a pretrained manner as “contextual word embeddings”